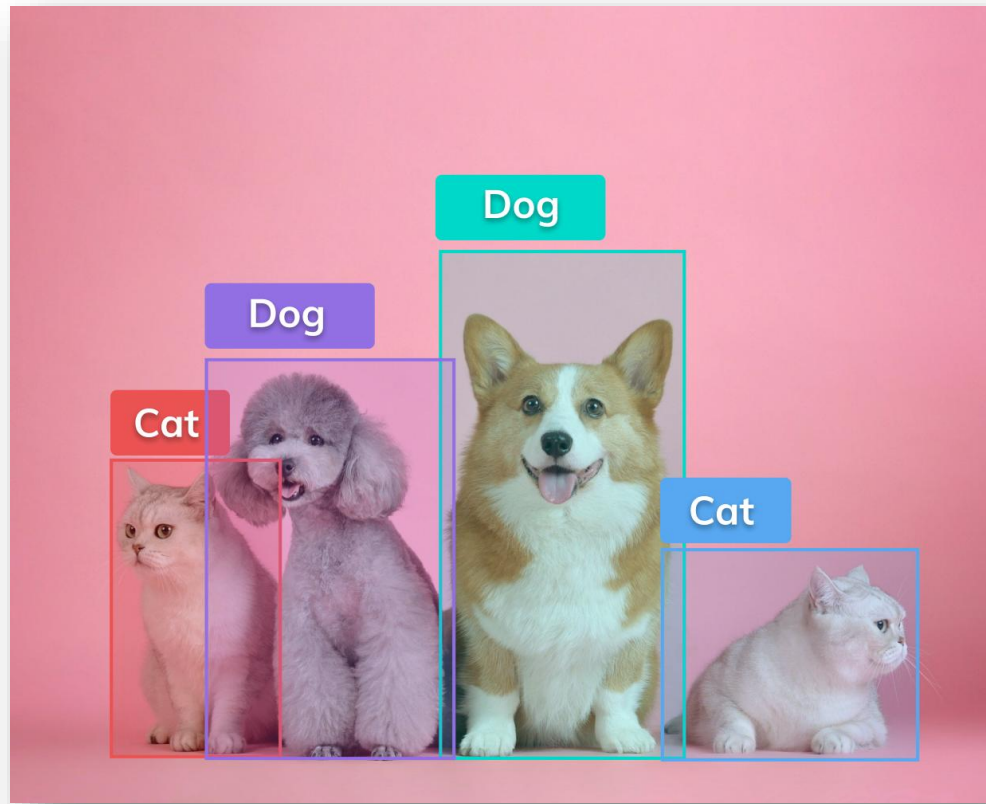




Object Detection

Introduction and Fundamentals

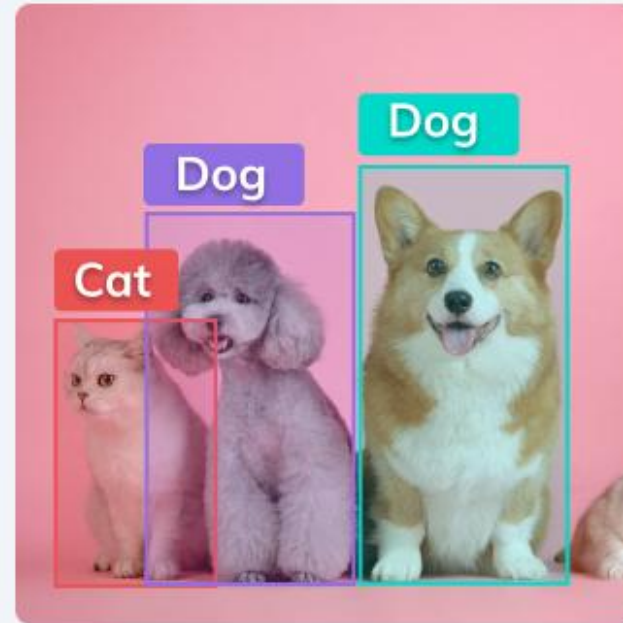
Image classification vs Object Detection

Classification



Cat

Detection



Cat, Dog, Dog

Overview

Classification



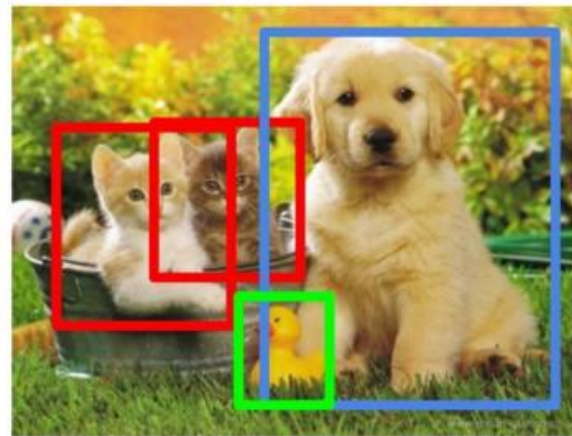
CAT

**Classification
+ Localization**



CAT

Object Detection



CAT, DOG, DUCK

**Instance
Segmentation**



CAT, DOG, DUCK

Single object

Multiple objects

Detection vs Segmentation

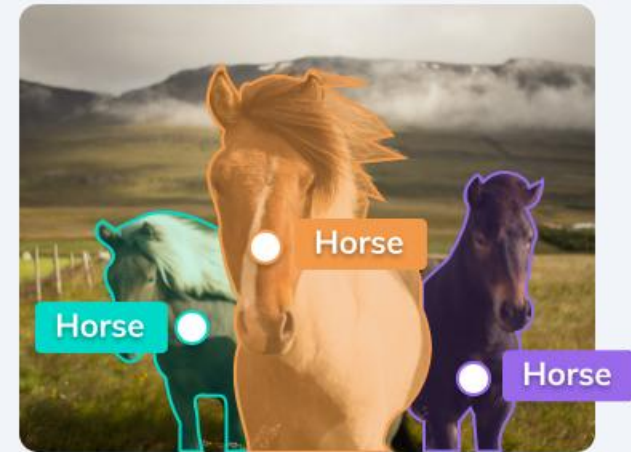
**Object Detection + Semantic Segmentation
= Instance Segmentation**



Object detection



Semantic Segmentation



Instance Segmentation

Classification, Detection and Segmentation

Is this a dog?



Image Classification

What is there in image
and where?



Object Detection

Which pixels belong to
which object?

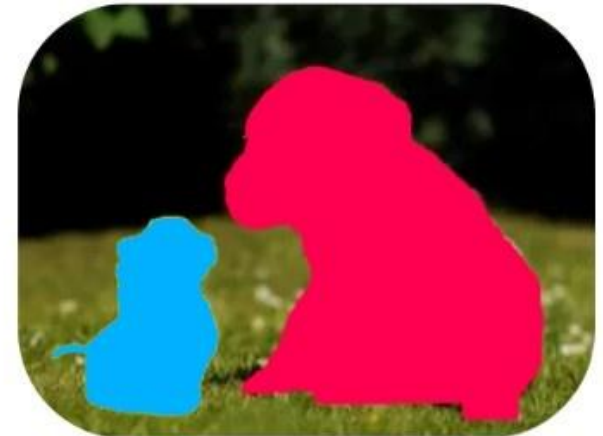


Image Segmentation

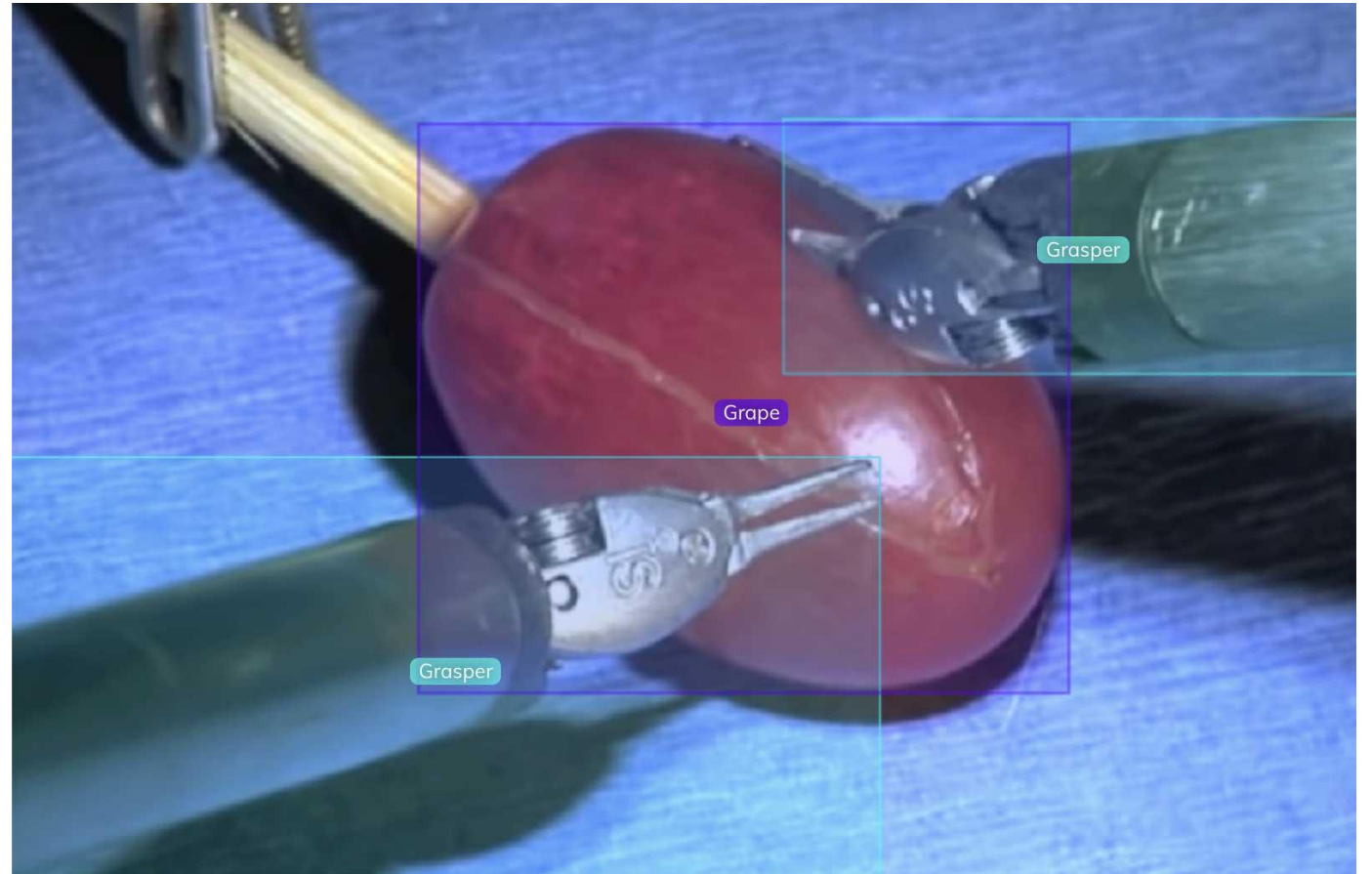
Object Detection Applications

- Autonomous vehicles



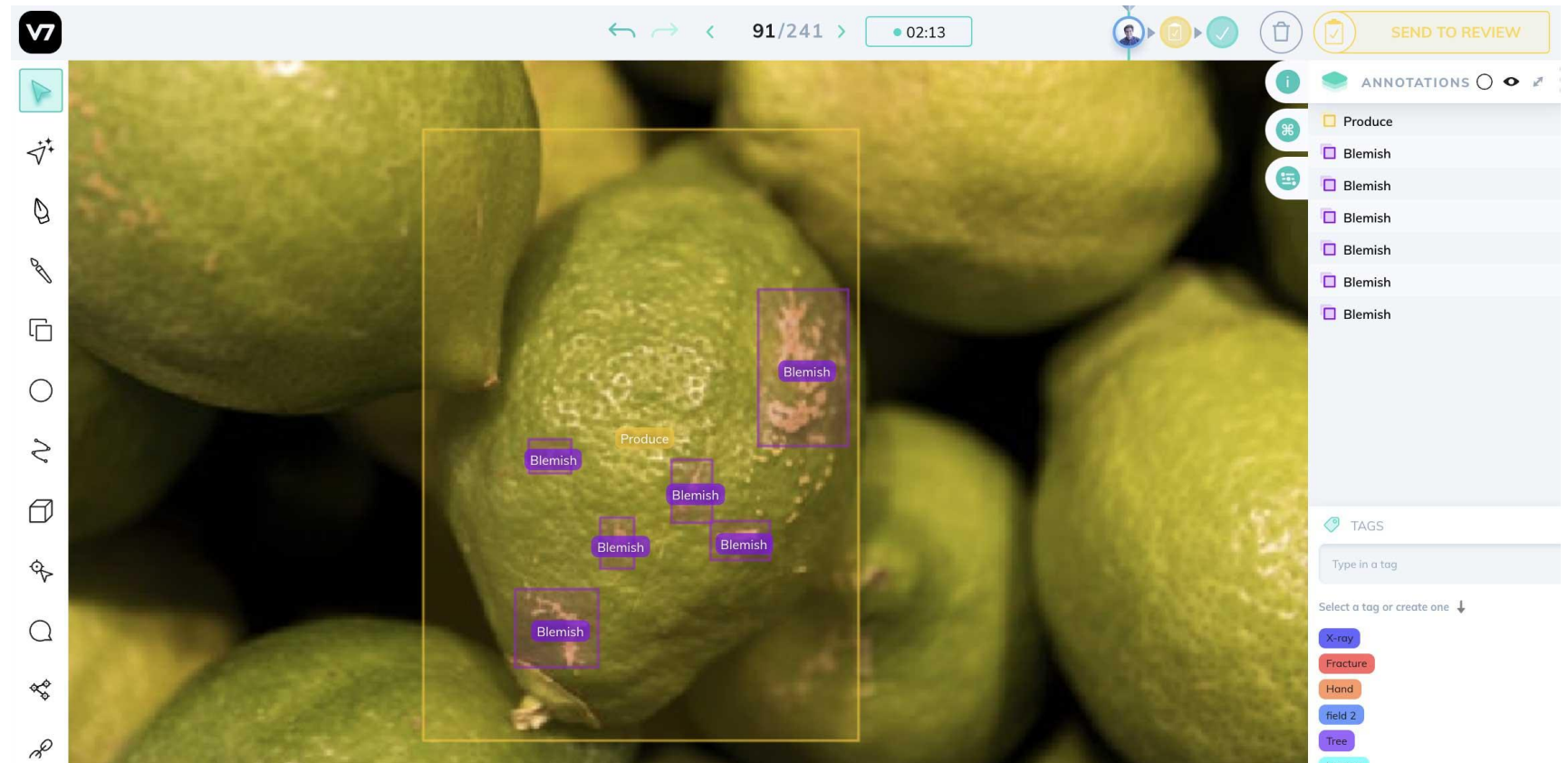
Object Detection Applications

- Intelligence video surgery



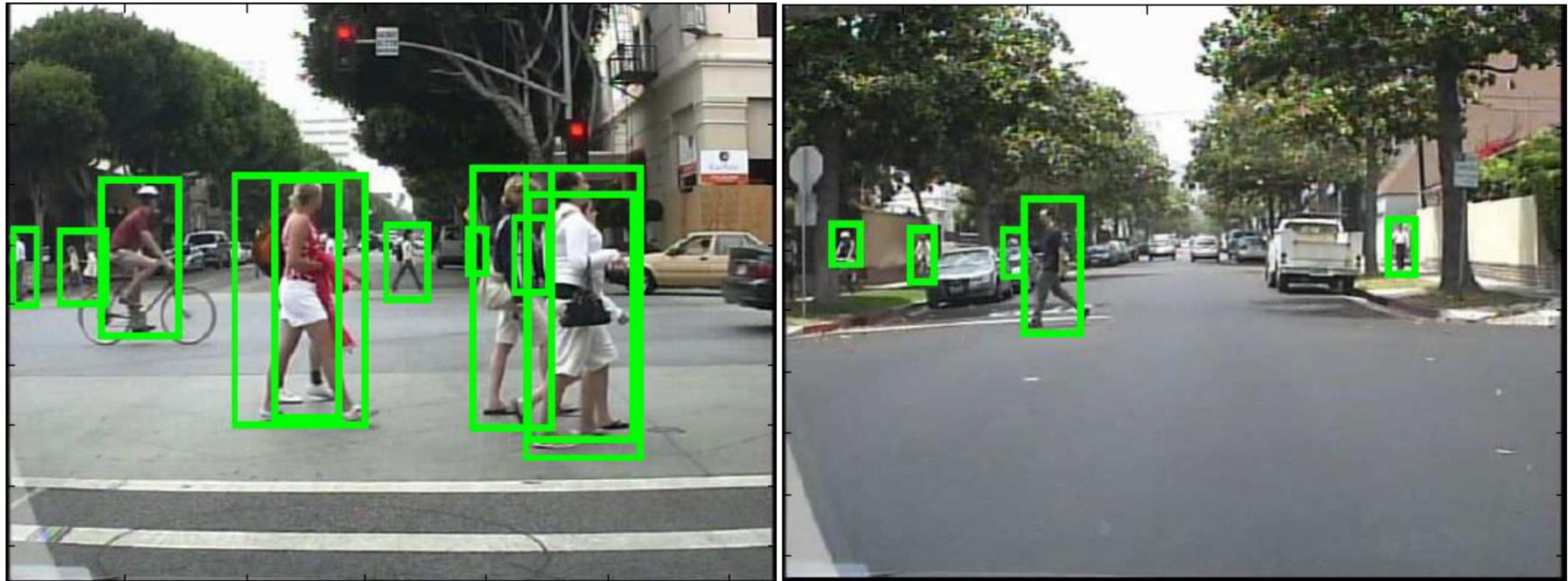
Object Detection Applications

- Defect Inspection



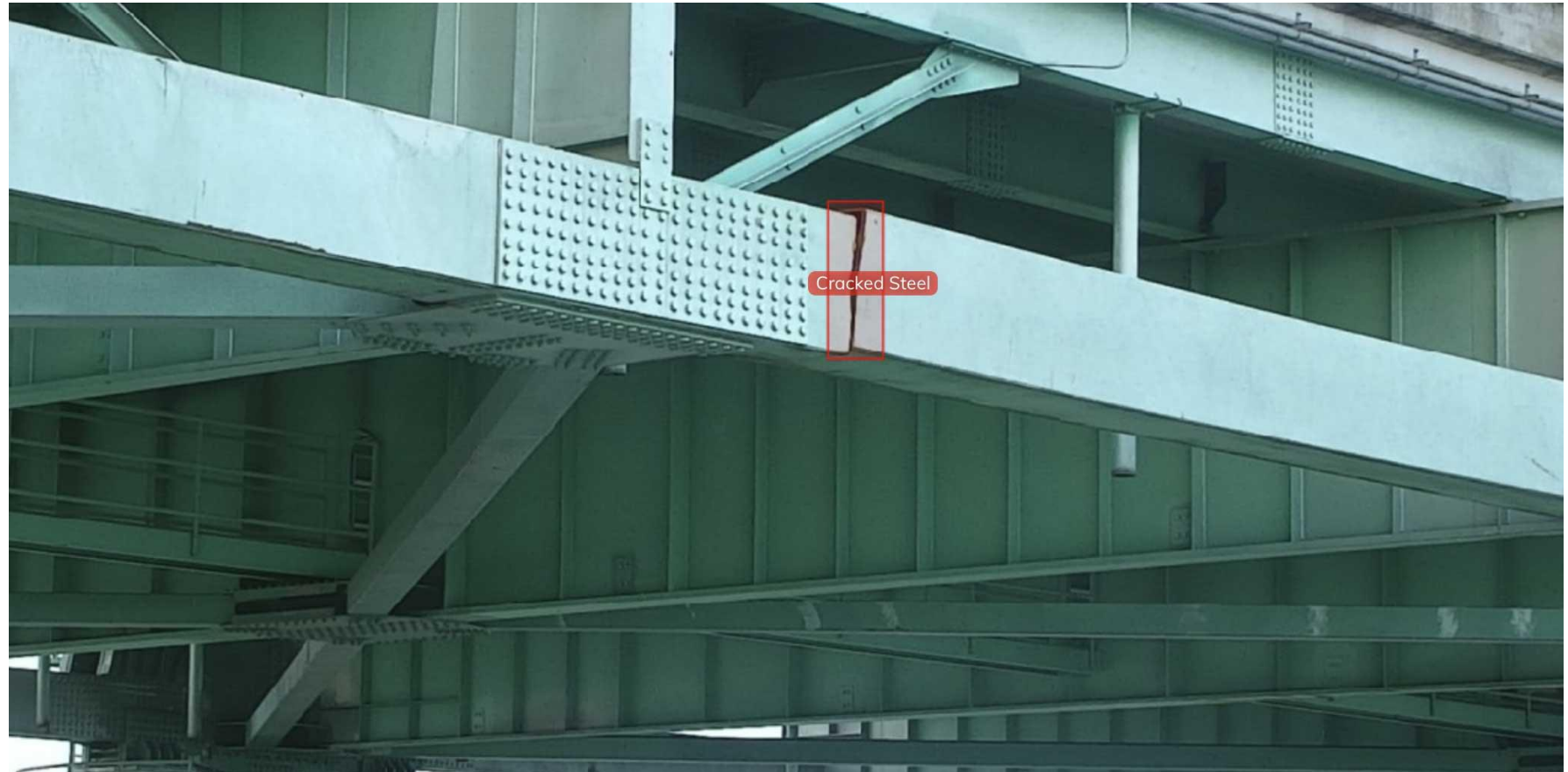
Object Detection Applications

- Pedestrian detection



Object Detection Applications

- AI Drone Navigation



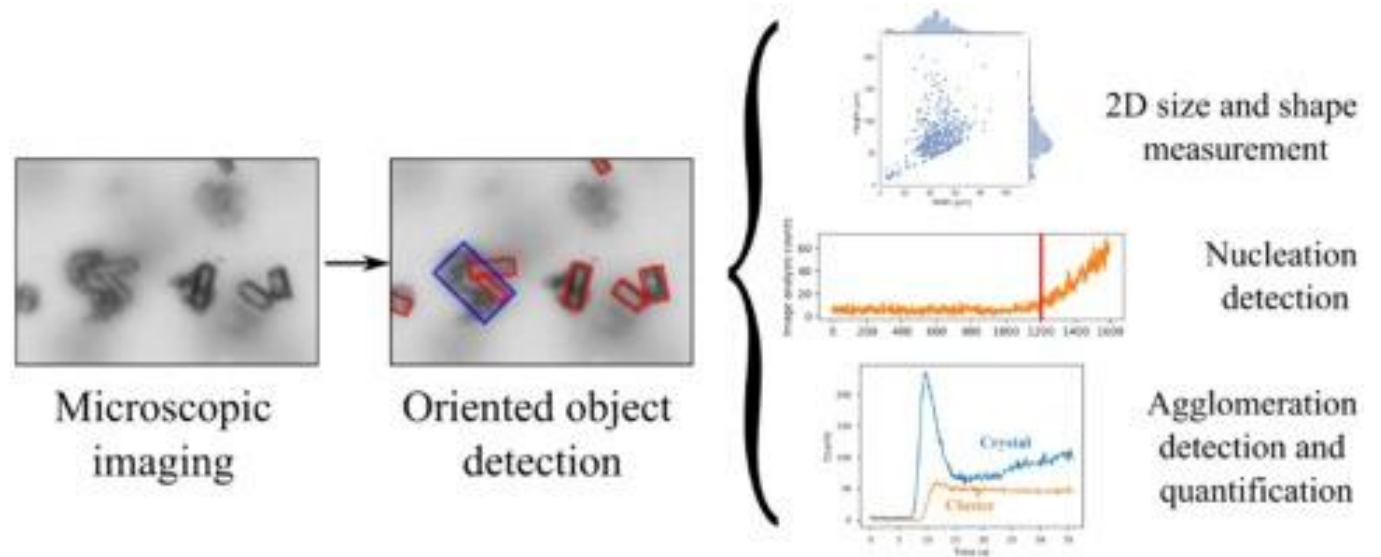
Object Detection Applications

- Camera surveillance



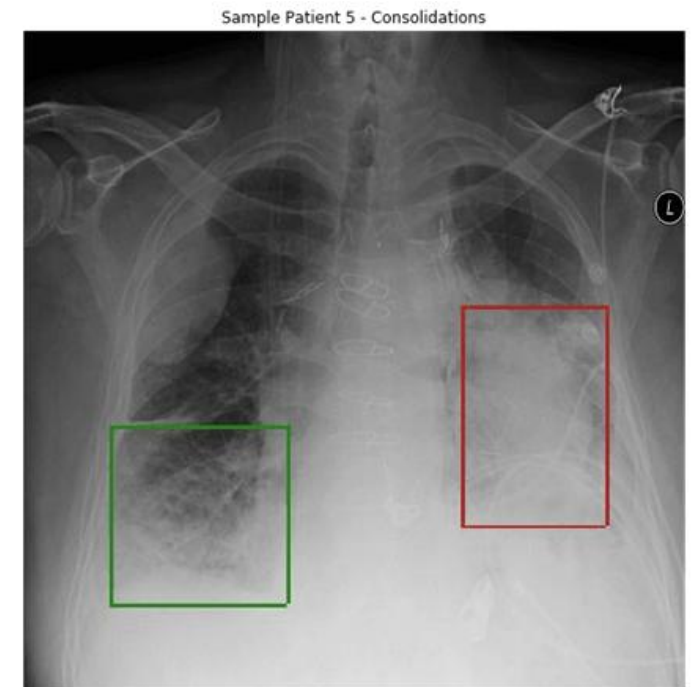
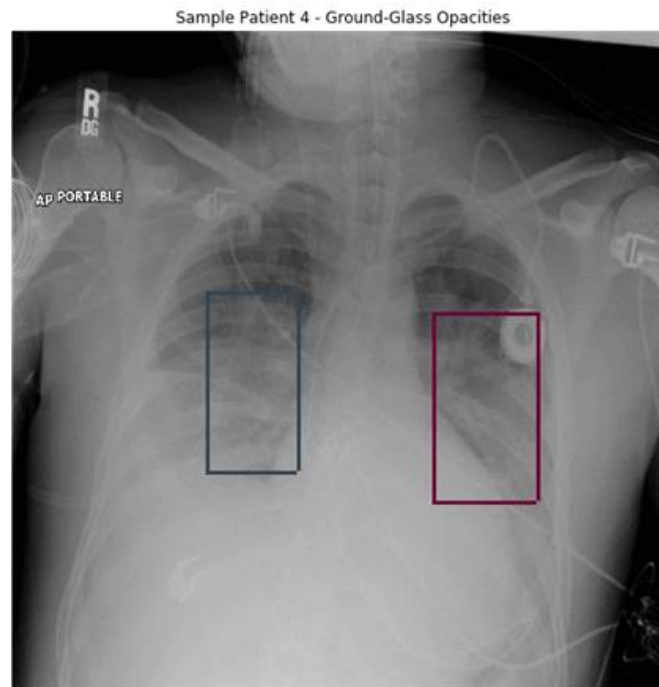
Object Detection Applications

- Science



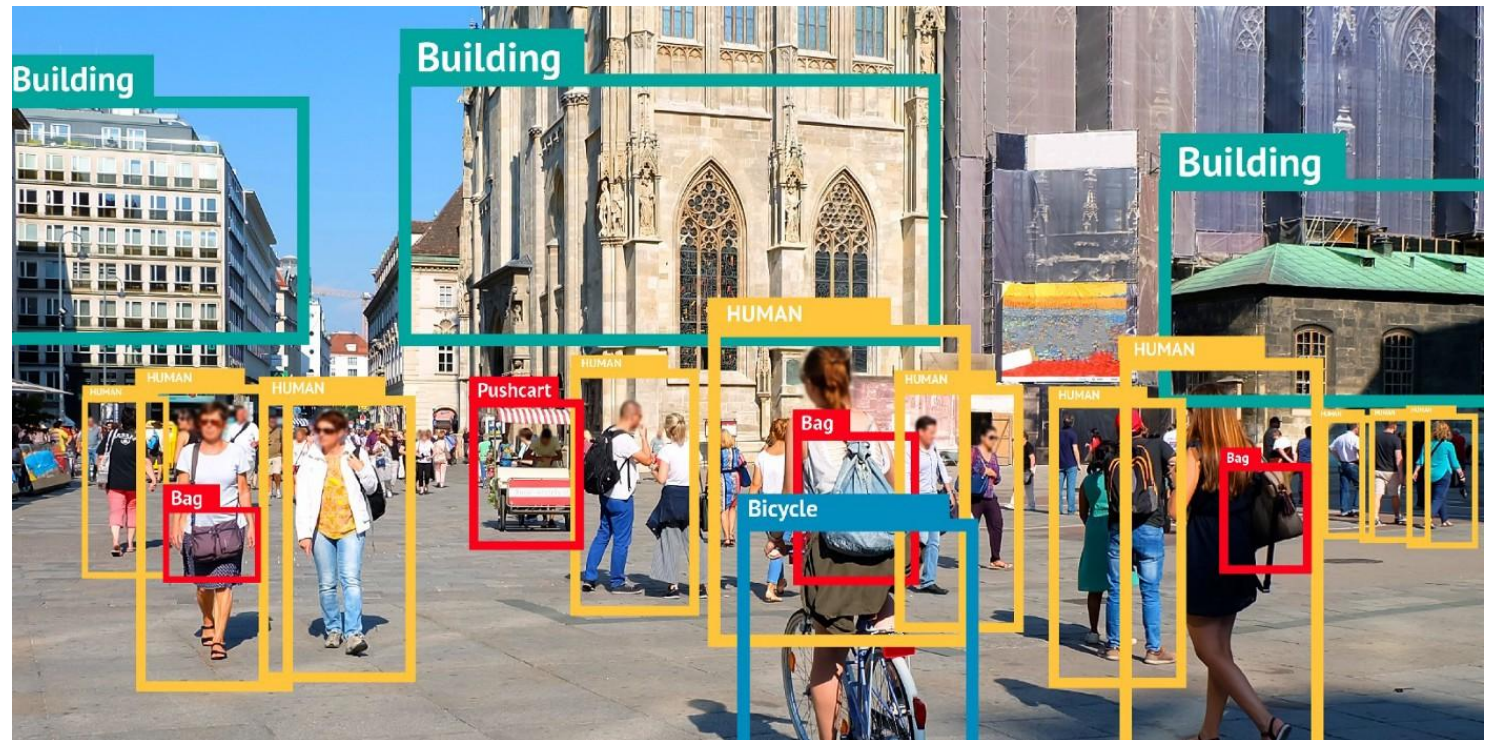
Object Detection Applications

- Radiology images



Object Detection Challenges

- Different size
- Different color and Texture and shape
- Different number of objects



Object Detection Challenges



Impact of Deep Learning

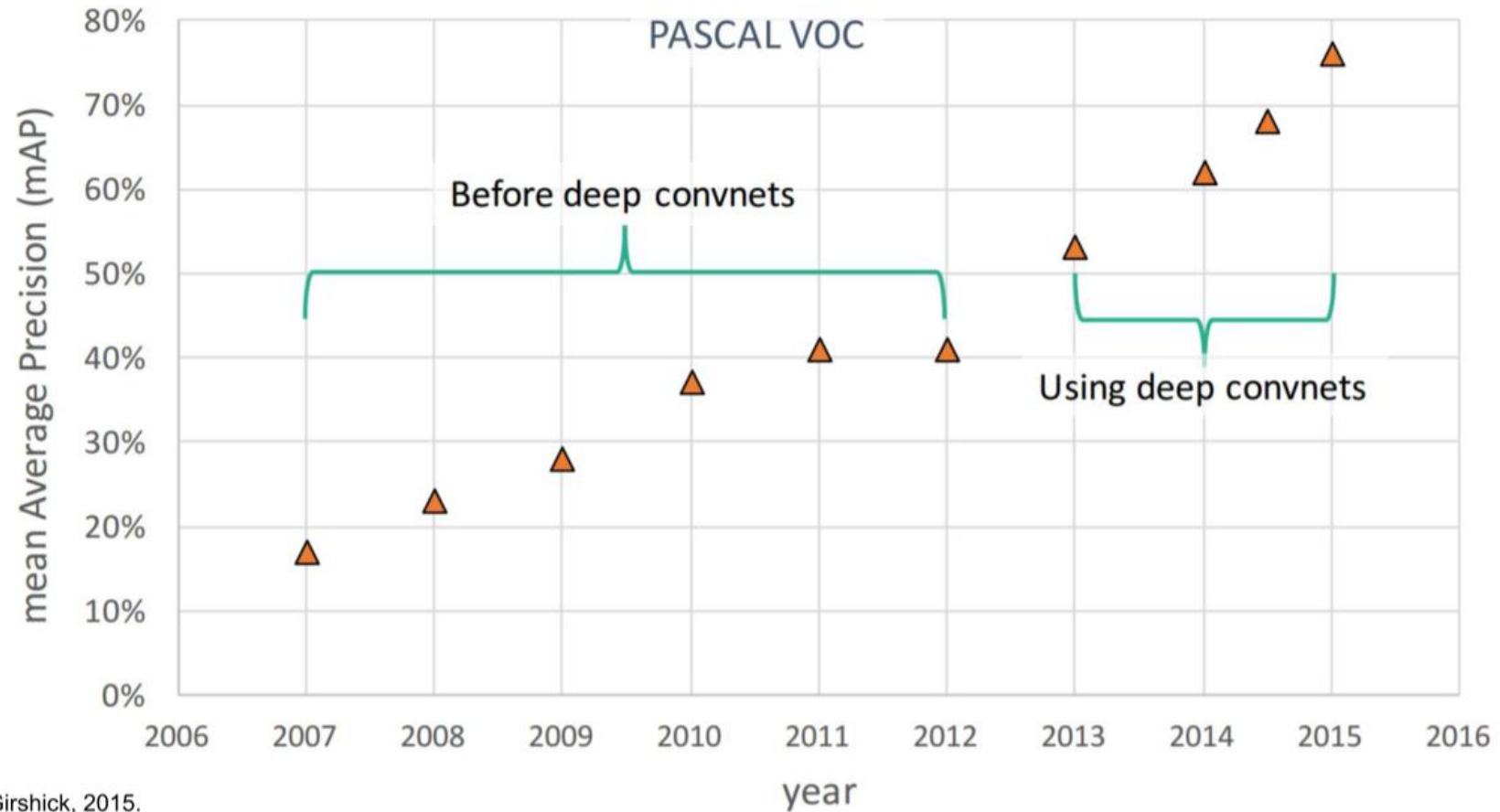
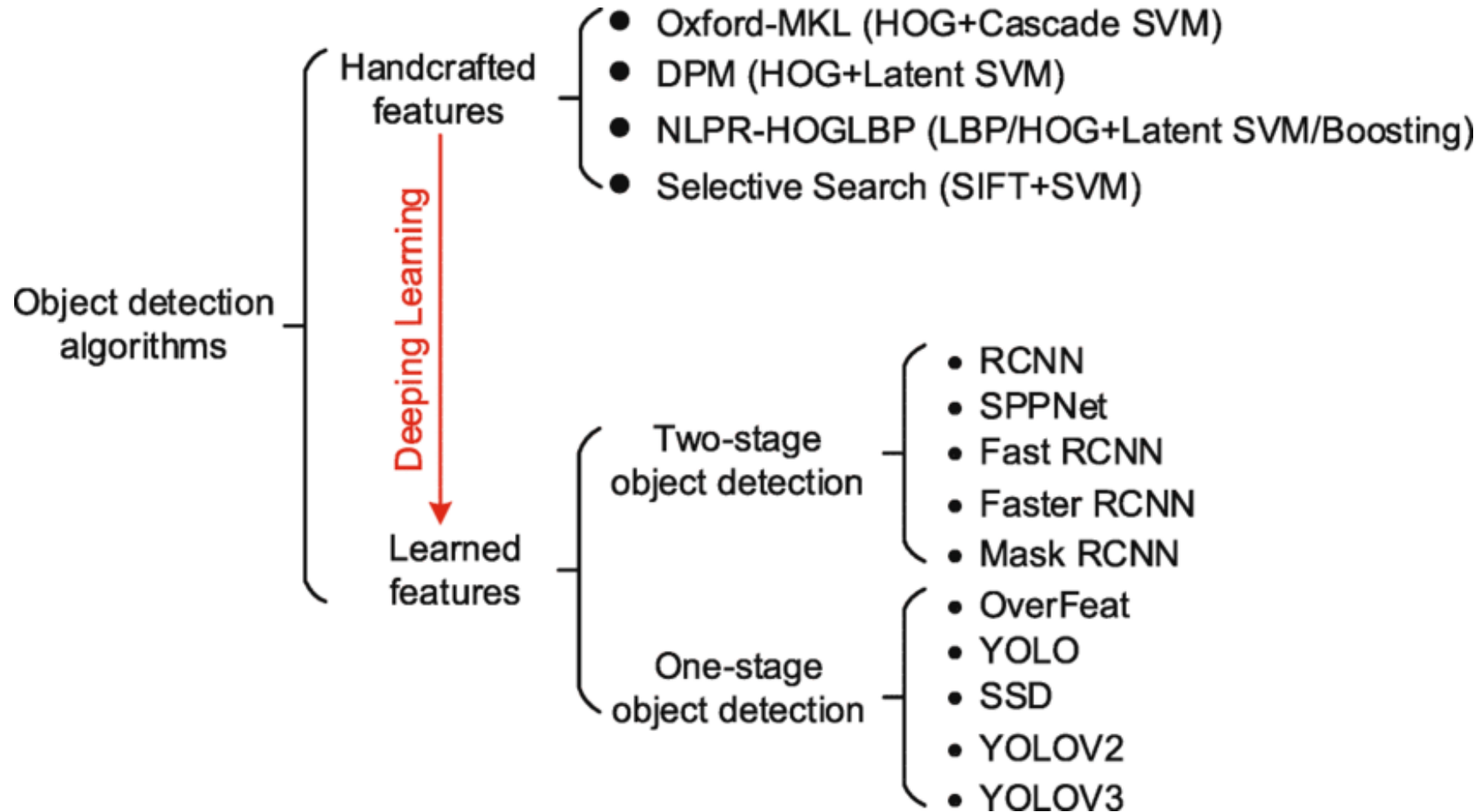


Figure copyright Ross Girshick, 2015.
Reproduced with permission.

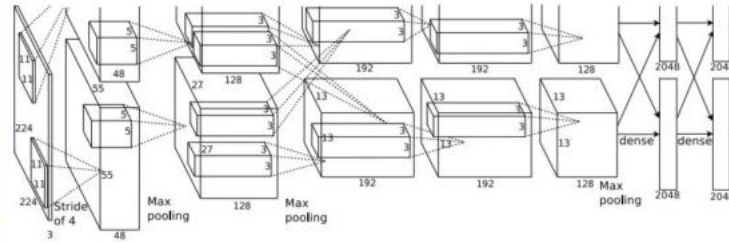
Object Detection Algorithms



Localization



[This image is CC0 public domain](#)



**Fully
Connected:**
4096 to 1000

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

Vector:
4096

**Fully
Connected:**
4096 to 4

**Box
Coordinates**
(x, y, w, h)

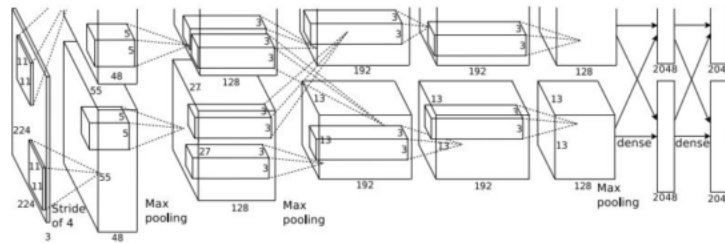
Treat localization as a
regression problem!

Localization

Classification + Localization



[This image is CC0 public domain](#)



Fully Connected:
4096 to 1000

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

Correct label:
Cat

Softmax Loss

Vector:
4096

Fully Connected:
4096 to 4

Box Coordinates
(x, y, w, h)

L2 Loss

Correct box:
(x', y', w', h')

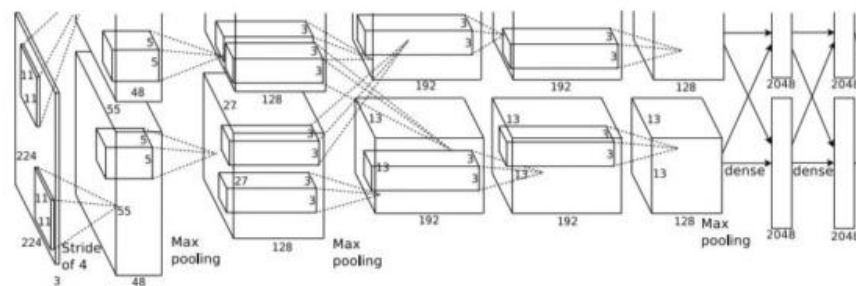
Treat localization as a regression problem!

(Human) Pose Estimation



Represent pose as a set of 14 joint positions:

- Left / right foot
- Left / right knee
- Left / right hip
- Left / right shoulder
- Left / right elbow
- Left / right hand
- Neck
- Head top



Vector:
4096

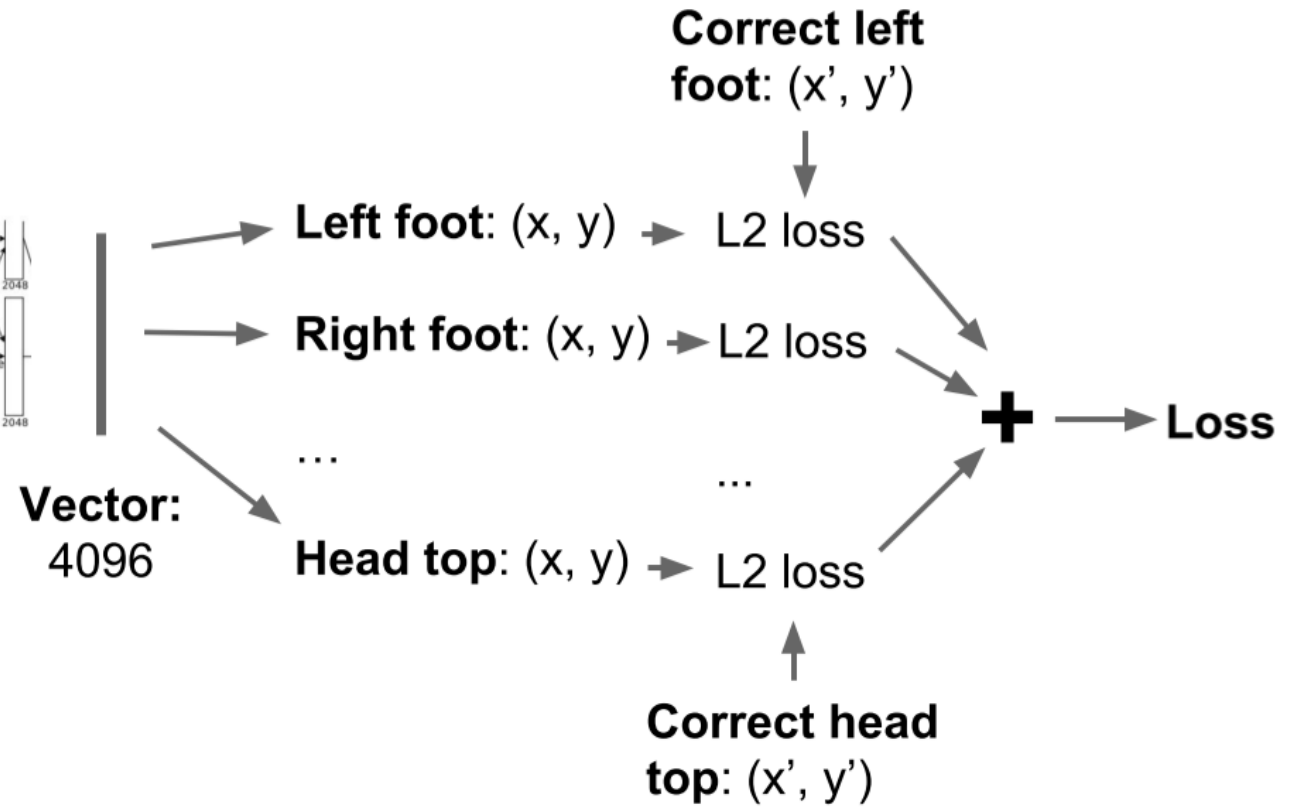
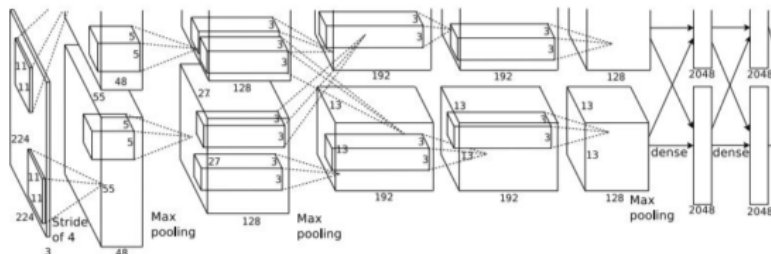
Left foot: (x, y)

Right foot: (x, y)

...

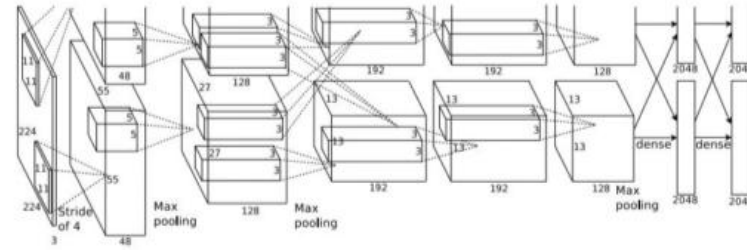
Head top: (x, y)

(Human) Pose Estimation

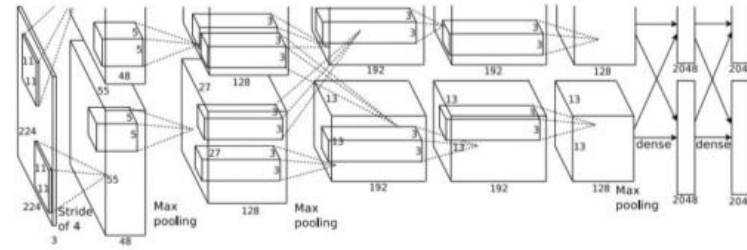


Toshev and Szegedy, "DeepPose: Human Pose Estimation via Deep Neural Networks", CVPR 2014

Object Detection as a Regression?



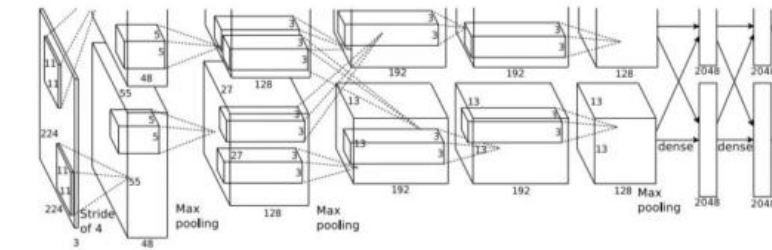
CAT: (x, y, w, h)



DOG: (x, y, w, h)

DOG: (x, y, w, h)

CAT: (x, y, w, h)



DUCK: (x, y, w, h)

DUCK: (x, y, w, h)

■ ■ ■

Datasets

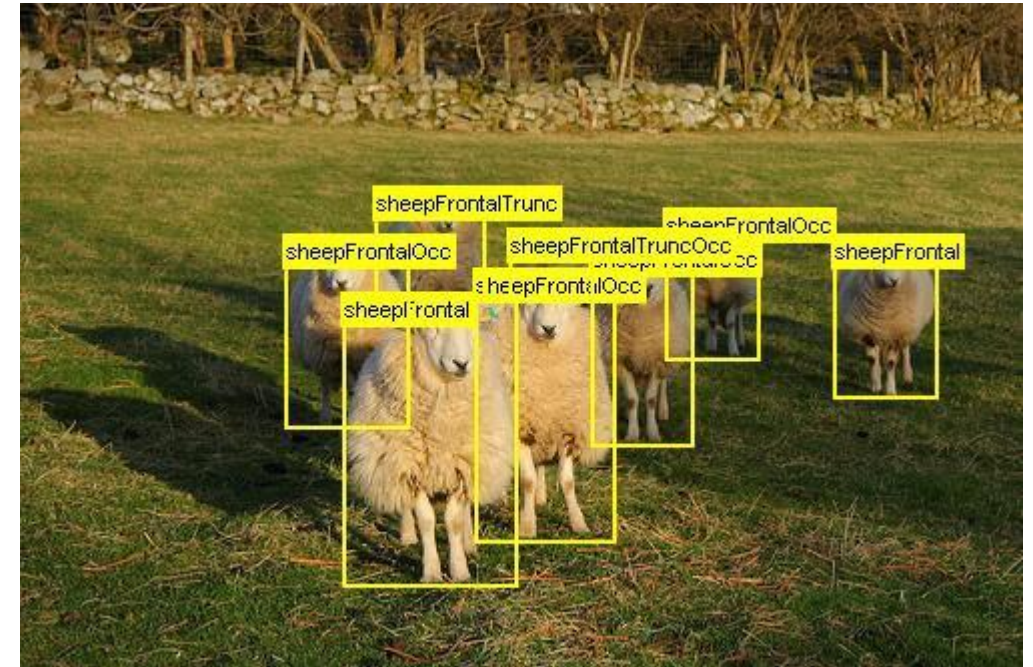
[Common Objects in Context \(COCO\)](#): COCO is a large-scale object detection, segmentation, and captioning dataset. It contains around 330,000 images out of which 200,000 are labelled for 80 different object categories.



Datasets

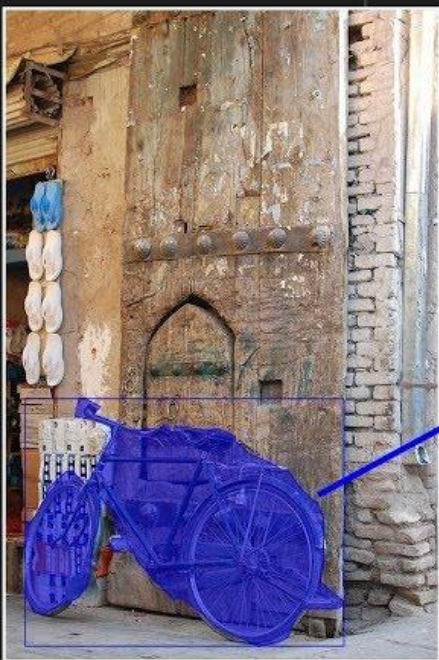
The PASCAL Visual Object Classes (VOC) 2012 dataset contains 20 object categories .

The **PASCAL VOC** dataset is split into three subsets: 1,464 images for training, 1,449 images for validation and a private testing set.



COCO dataset format

COCO DATASET FORMAT



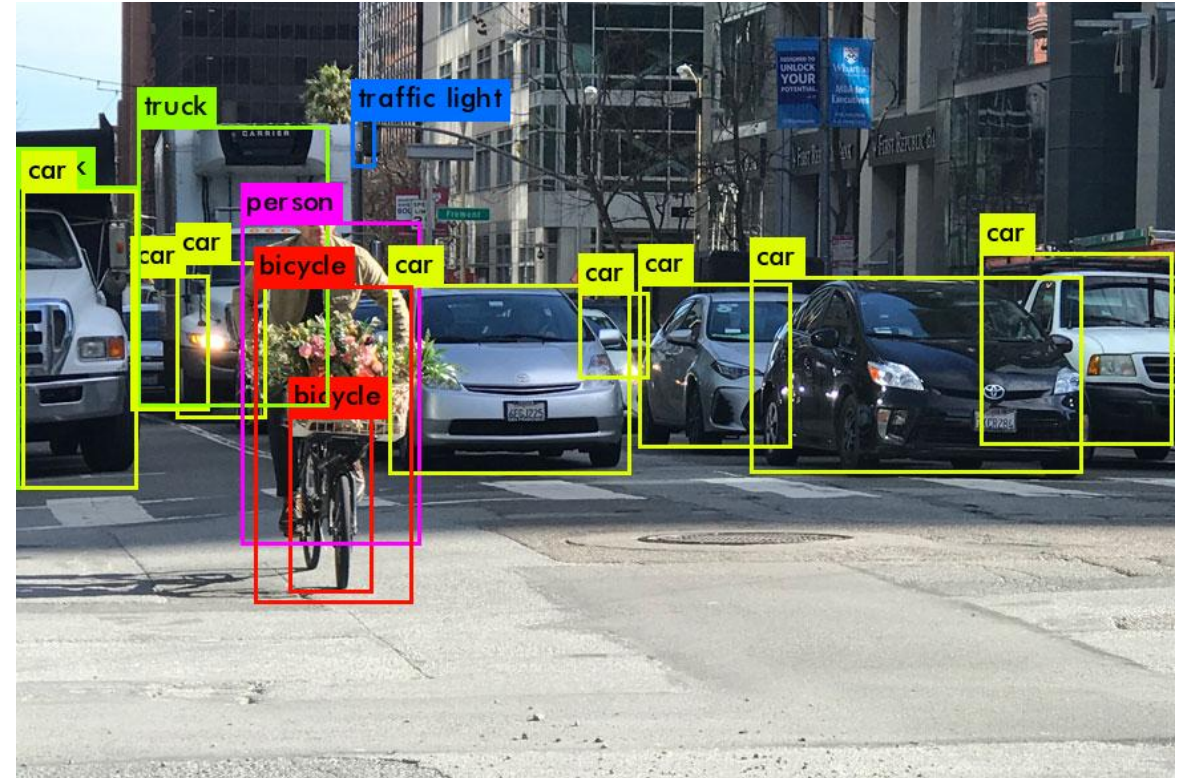
```
"annotations": [  
  {  
    "id": 125686,  
    "category_id": 2,  
    "iscrowd": 0,  
    "segmentation": [[164.81, 417.51, 164.81, 417.51],  
    "image_id": 242287,  
    "area": 42061.80340000001,  
    "bbox": [19.23, 383.18, 314.5, 244.46]  
  },  
  {  
    "id": 1409619,  
    "category_id": 2,  
    "iscrowd": 0,  
    "segmentation": [[164.81, 417.51, 164.81, 417.51],  
    "image_id": 242287,  
    "area": 42061.80340000001,  
    "bbox": [19.23, 383.18, 314.5, 244.46]  
  }  
]
```


Pascal voc dataset format

```
Sachin.xml x
1 <annotation verified="yes">
2   <folder>Annotation</folder>
3   <filename>Sachin.jpg</filename>
4   <path>Cricketer-PascalVOC-export/Annotations/Sachin.jpg</path>
5   <source>
6     <database>Unknown</database>
7   </source>
8   <size>
9     <width>1024</width>
10    <height>672</height>
11    <depth>3</depth>
12  </size>
13  <segmented>0</segmented>
14  <object>
15    <name>cricketers</name>
16    <pose>Unspecified</pose>
17    <truncated>0</truncated>
18    <difficult>0</difficult>
19    <bndbox>
20      <xmin>197.0057899090157</xmin>
21      <ymin>121.13067465321564</ymin>
22      <xmax>683.4342431761786</xmax>
23      <ymax>619.3841424968474</ymax>
24    </bndbox>
25  </object>
26 </annotation>
27
```

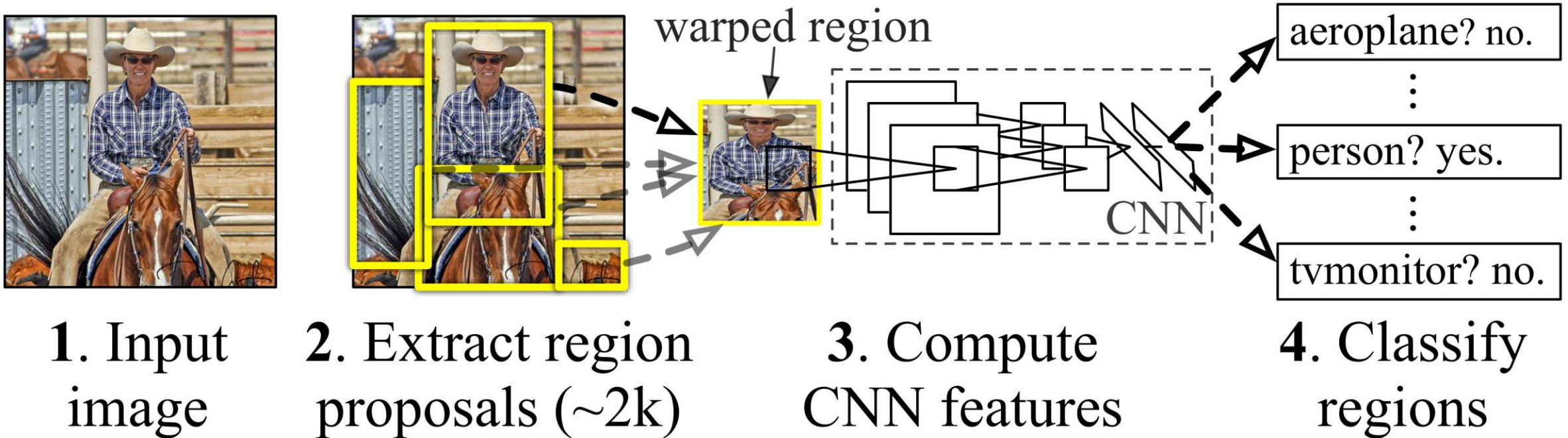
Algorithms

- RCNN (2014)
- Fast RCNN (2015)
- Faster RCNN (2016)
- YOLO (2016) (v1,v2,v3,v4,v5,v6,v7,v8)
- SSD (2016)

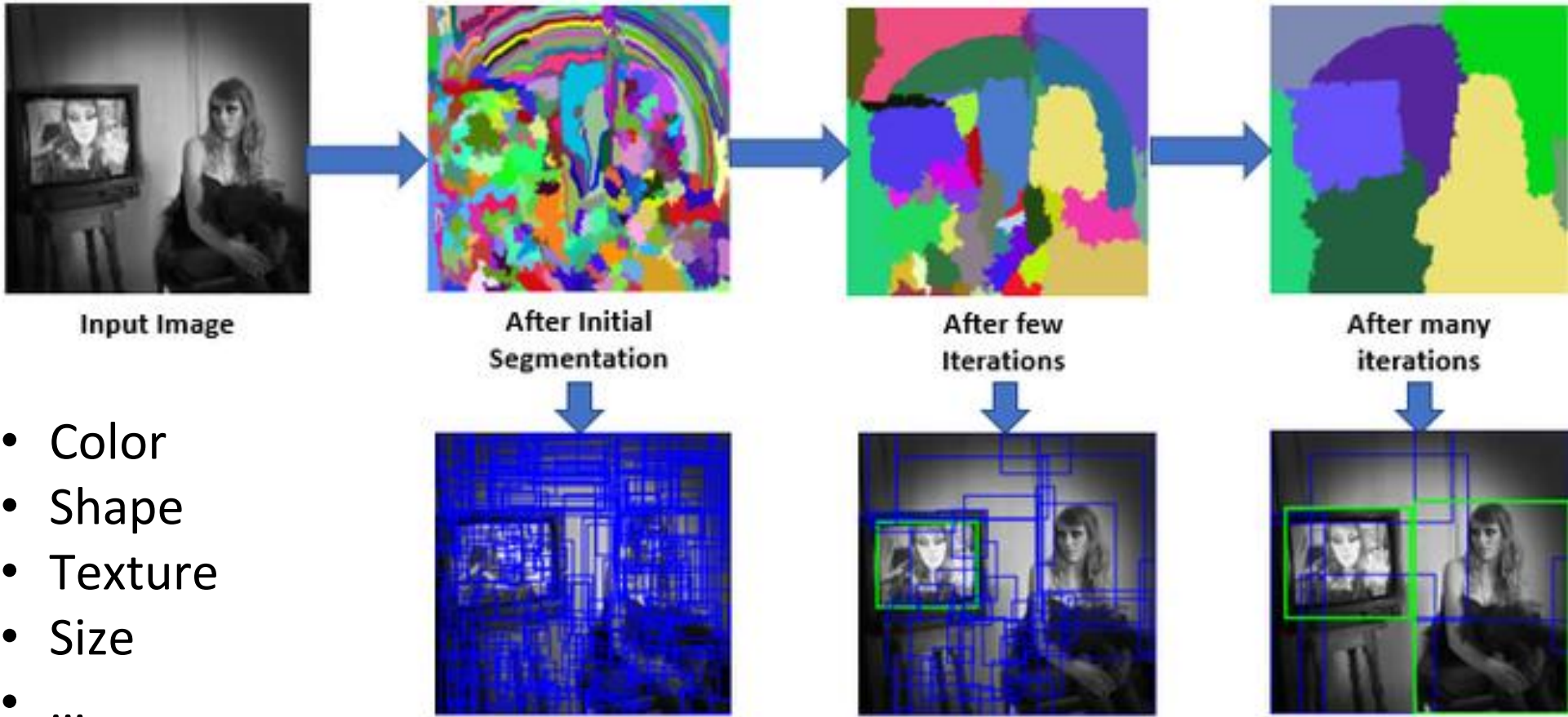


RCNN

R-CNN: *Regions with CNN features*

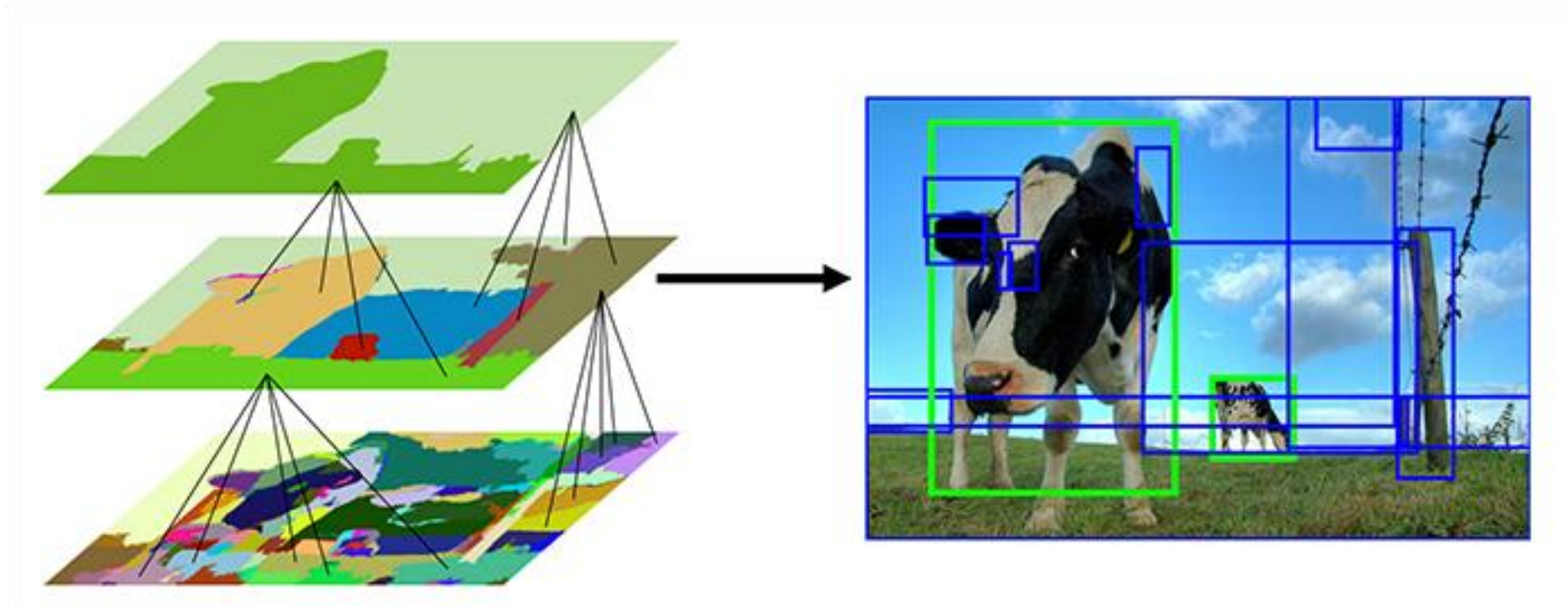


Selective Search Method

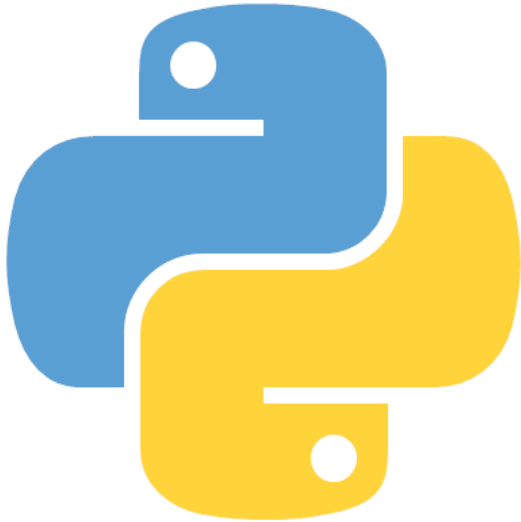


- Color
- Shape
- Texture
- Size
- ...

Selective Search Method



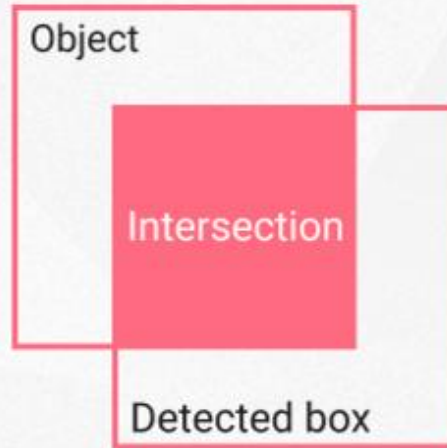
Coding Session



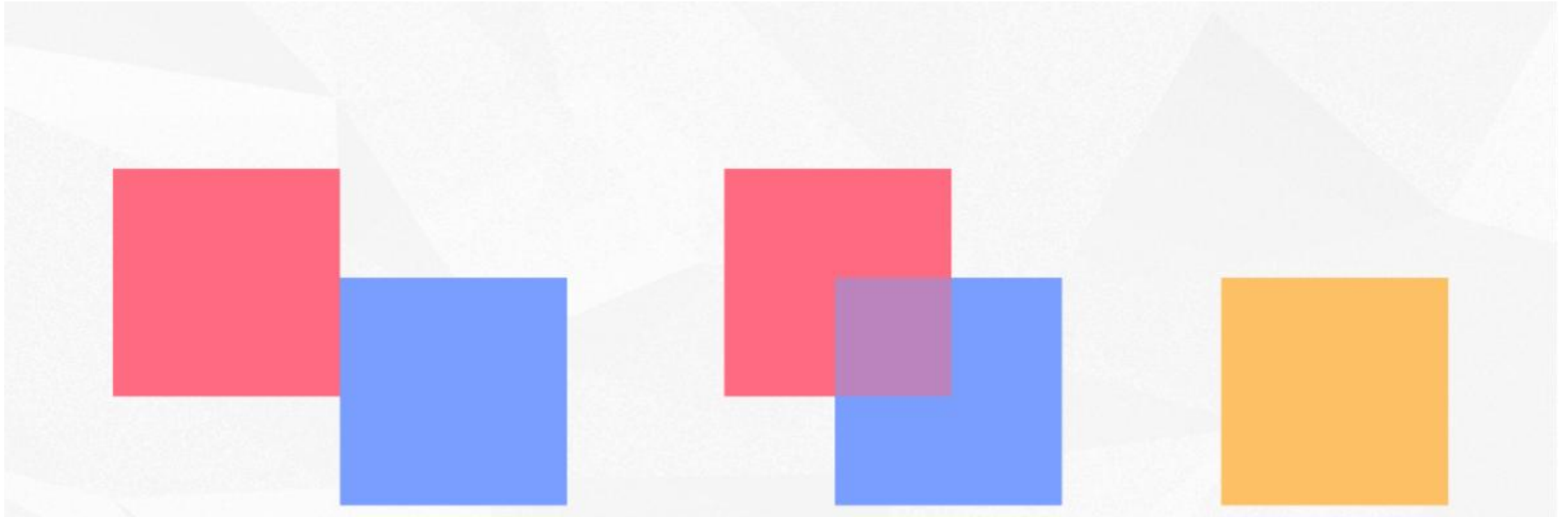
Selective Search

The Intersection over Union (IoU)

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

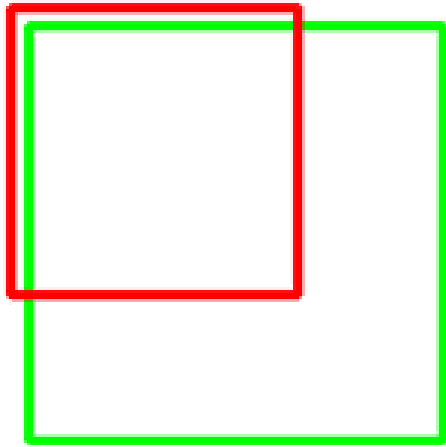


IoU



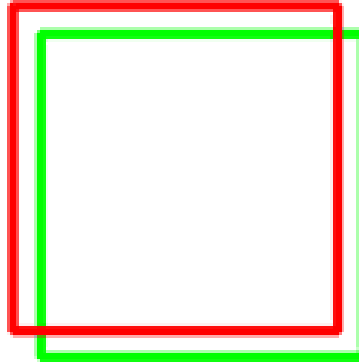
IoU in papers

IoU: 0.4034



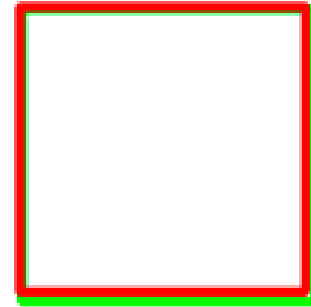
Poor

IoU: 0.7330



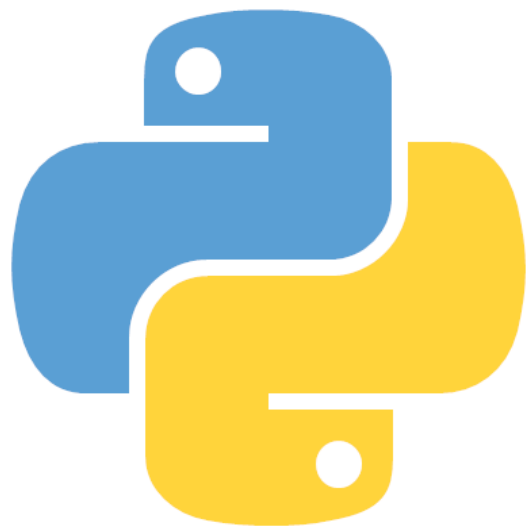
Good

IoU: 0.9264

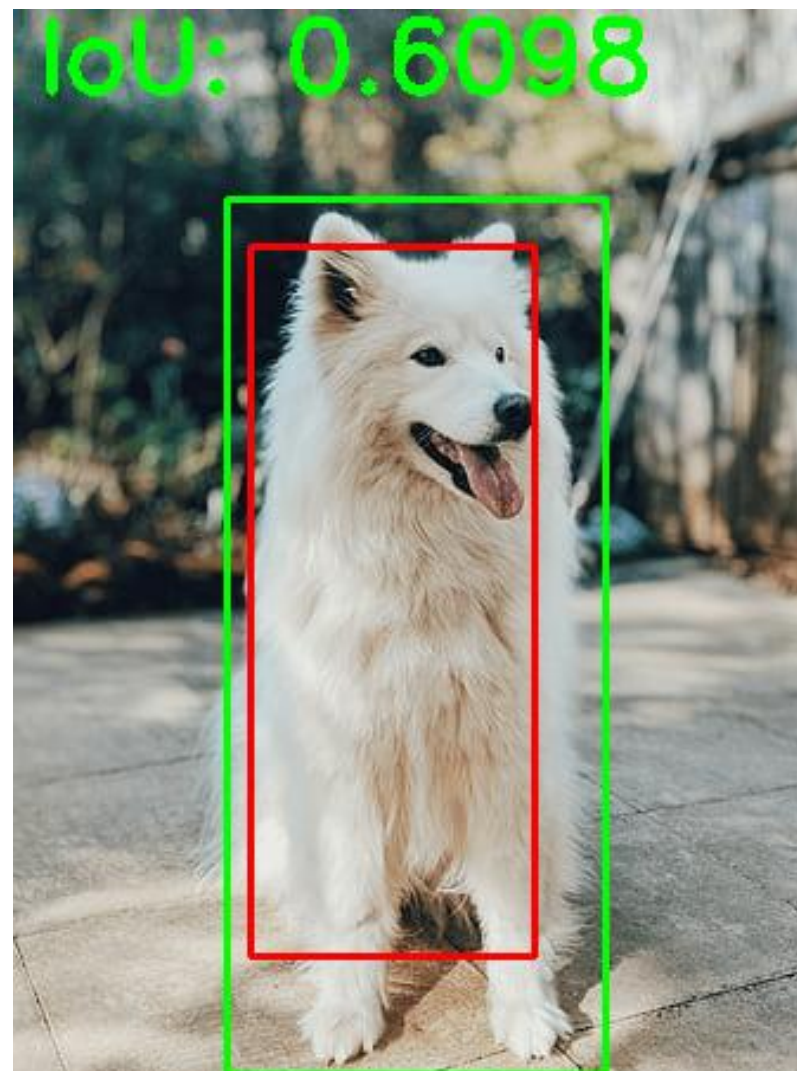


Excellent

Coding Session

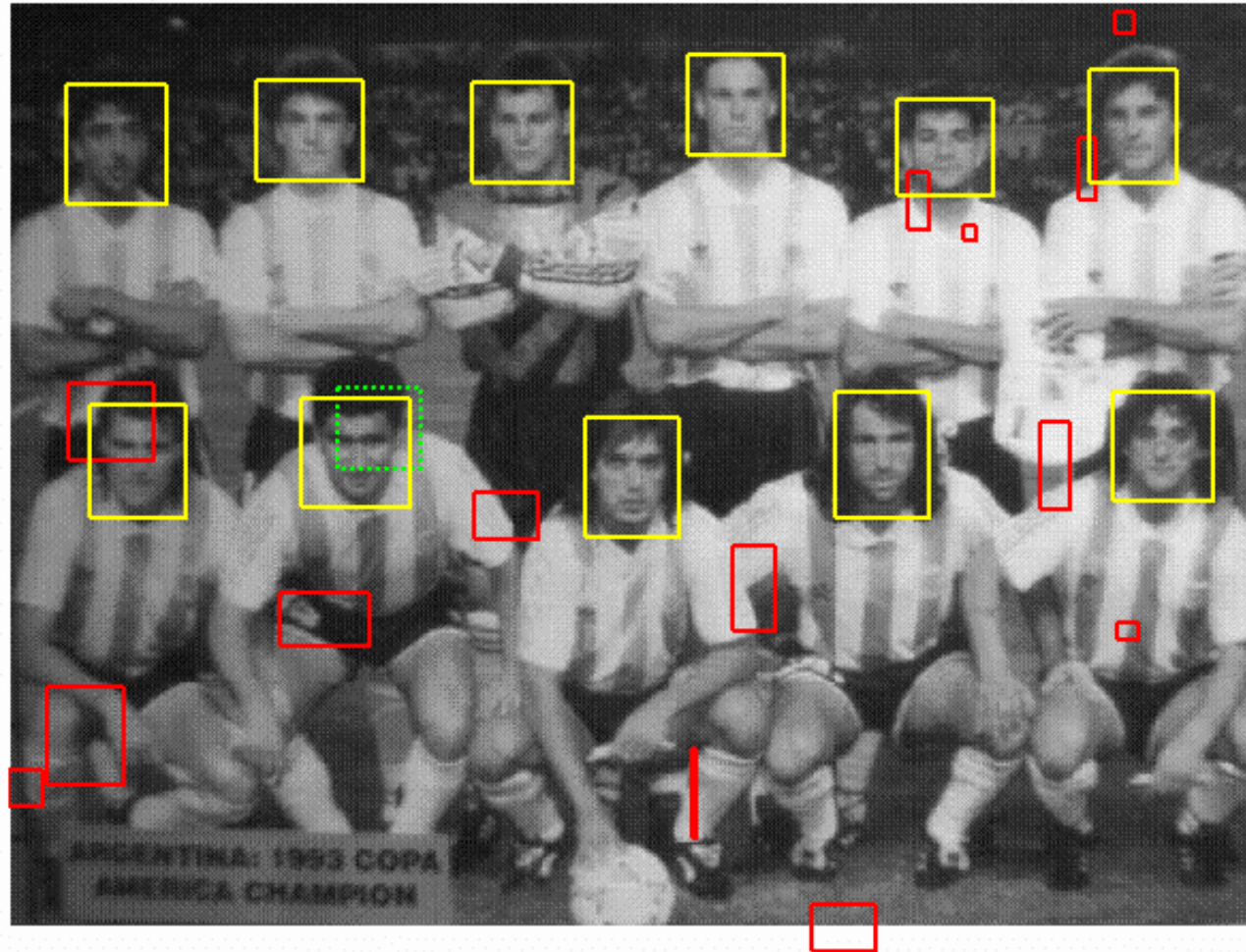


IOU



False positive in Object Detection

image: "Argentina.jpg" (green=true pos, red=false pos, yellow=ground truth), 1/11 found



Mean Average Precision (mAP)

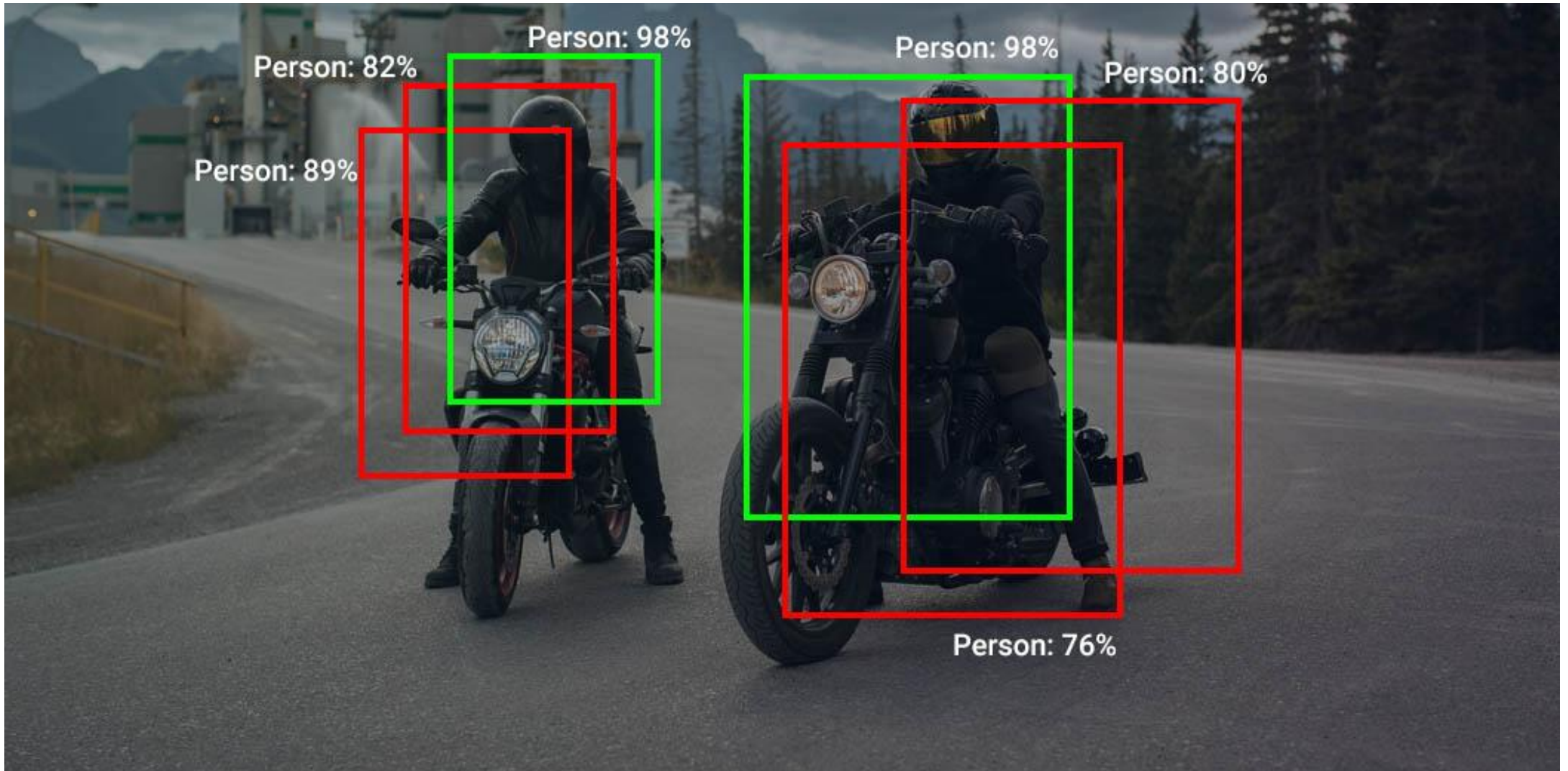

$$mAP = \frac{1}{|classes|} \sum_{c \in classes} \frac{|TP_c|}{|FP_c| + |TP_c|}$$

mAP in papers

Model NO.	Basic Feature Extraction Network	Model Parameters		Detection Box Retention Algorithm	Testing Dataset A mAP (%)	Testing Dataset B mAP (%)
		Basic Anchor Size (pixels)	Pooling Type of Basic Feature Extraction Network			
0	VGG16	$8 \times 16, 16 \times 16, 32 \times 16$	max-pooling	NMS	83.1	85.2
1	ResNet50	$8 \times 16, 16 \times 16, 32 \times 16$	average-pooling	NMS	85.5	86.6
2	ResNet50	$4 \times 16, 8 \times 16, 16 \times 16$	average-pooling	NMS	86.0	87.3
3	ResNet50	$8 \times 16, 16 \times 16, 32 \times 16$	max-pooling	NMS	85.9	87.1
4	ResNet50	$4 \times 16, 8 \times 16, 16 \times 16$	max-pooling	NMS	86.9	88.7
5	ResNet50	$8 \times 16, 16 \times 16, 32 \times 16$	average-pooling	Soft-NMS	87.7	88.9
6	ResNet50	$4 \times 16, 8 \times 16, 16 \times 16$	average-pooling	Soft-NMS	88.3	89.6
7	ResNet50	$4 \times 16, 8 \times 16, 16 \times 16$	max-pooling	Soft-NMS	89.3	90.7
8	ResNet50	$8 \times 16, 16 \times 16, 32 \times 16$	max-pooling	Soft-NMS	88.2	89.1

Note: Testing Dataset A represents the original dataset without augmentation, and Testing Dataset B represents the expanded dataset using augmentation. All parameter settings and discussions in this paper are only performed on Dataset B.

Non Maximum Suppression (NMS)



NMS

Why we need it?

Once the detector outputs a **large number of bounding boxes**, it is necessary to **filter out the best ones**

Input

We get a list P of prediction BBoxes of the form $(x1, y1, x2, y2, c)$, where $(x1, y1)$ and $(x2, y2)$ are the ends of the BBox and c is the predicted confidence score of the model. We also get overlap threshold $IoU\ thresh_iou$.

Output

We return a list keep of filtered prediction BBoxes.

NMS, in a nutshell

In layman terms, we select the predictions with the **maximum confidence** and **suppress all the other predictions** having overlap with the selected predictions greater than a threshold.

In other words, we **take the maximum and suppress the non-maximum ones**, hence the name non-maximum suppression.

Output of NMS

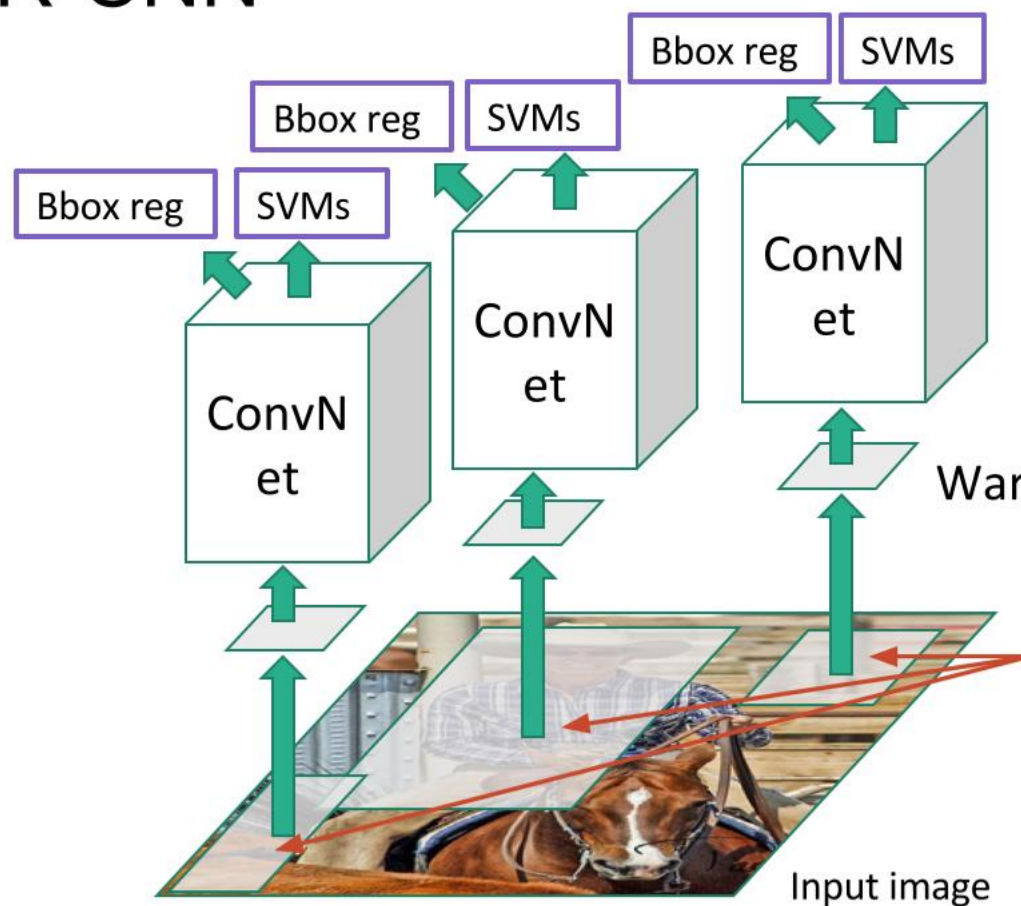


Exercise

Soft-NMS ?

RCNN

R-CNN



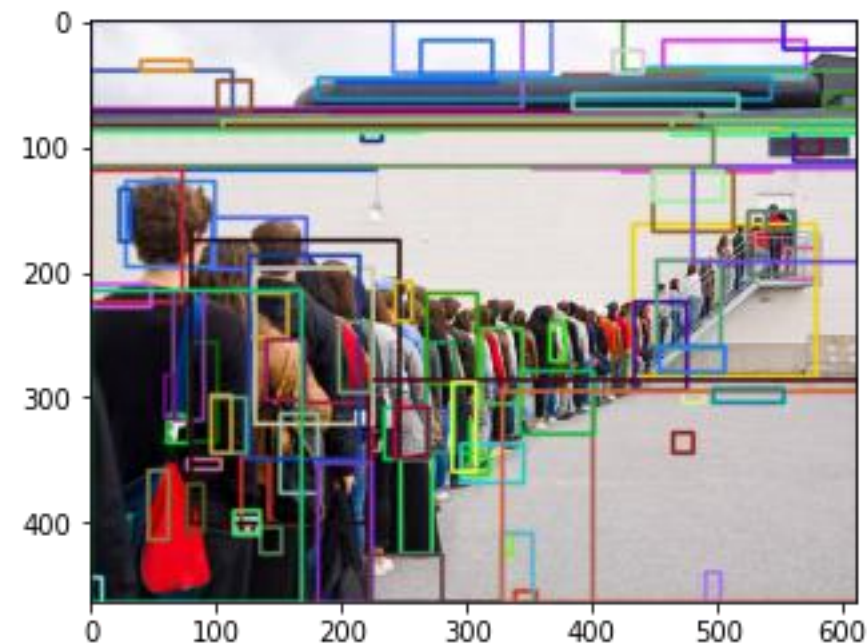
Linear Regression for bounding box offsets

Classify regions with SVMs

Forward each region through ConvNet

Warped image regions

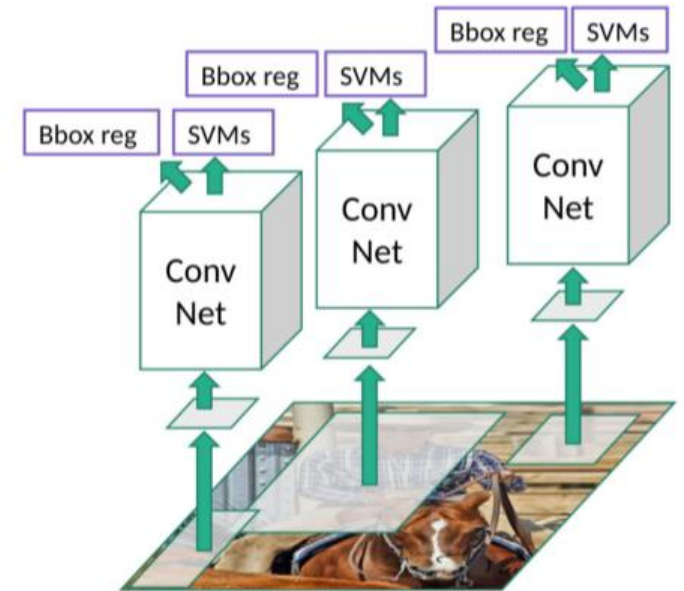
Regions of Interest (RoI) from a proposal method (~2k)



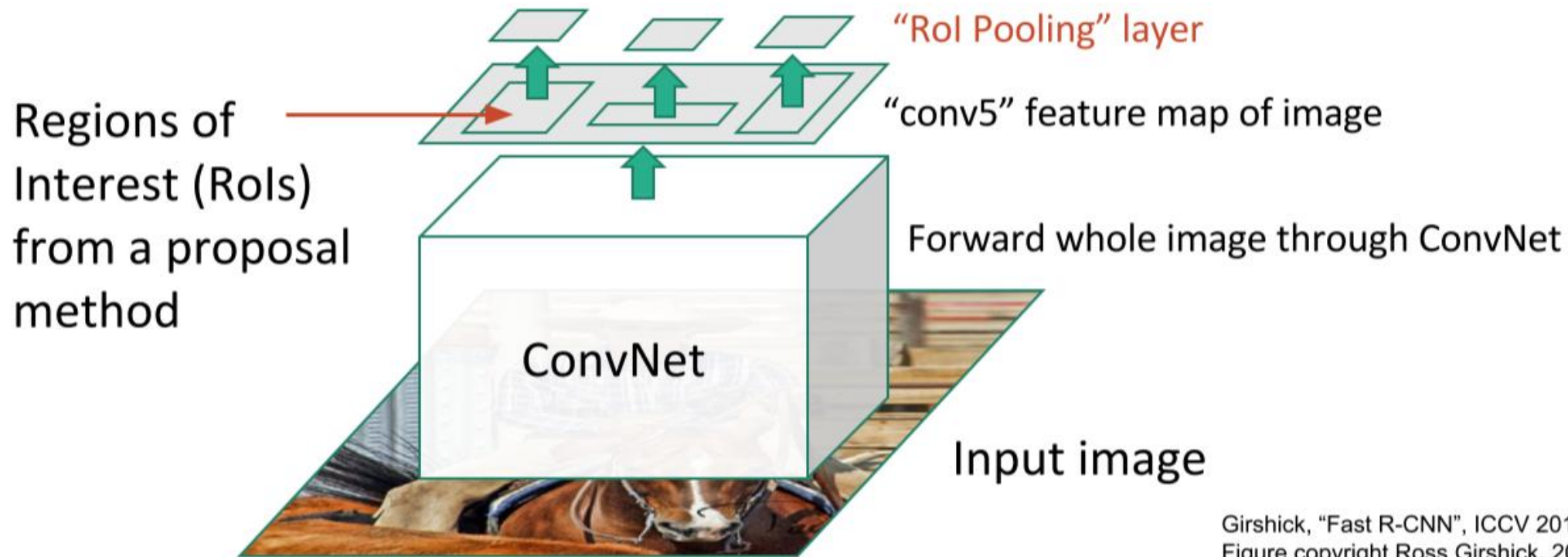
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN Problems

- Ad hoc training objectives
 - Fine-tune network with softmax classifier (log loss)
 - Train post-hoc linear SVMs (hinge loss)
 - Train post-hoc bounding-box regressions (least squares)
- Training is slow (84h), takes a lot of disk space
- Inference (detection) is slow
 - 47s / image with VGG16 [Simonyan & Zisserman. ICLR15]
 - Fixed by SPP-net [He et al. ECCV14]

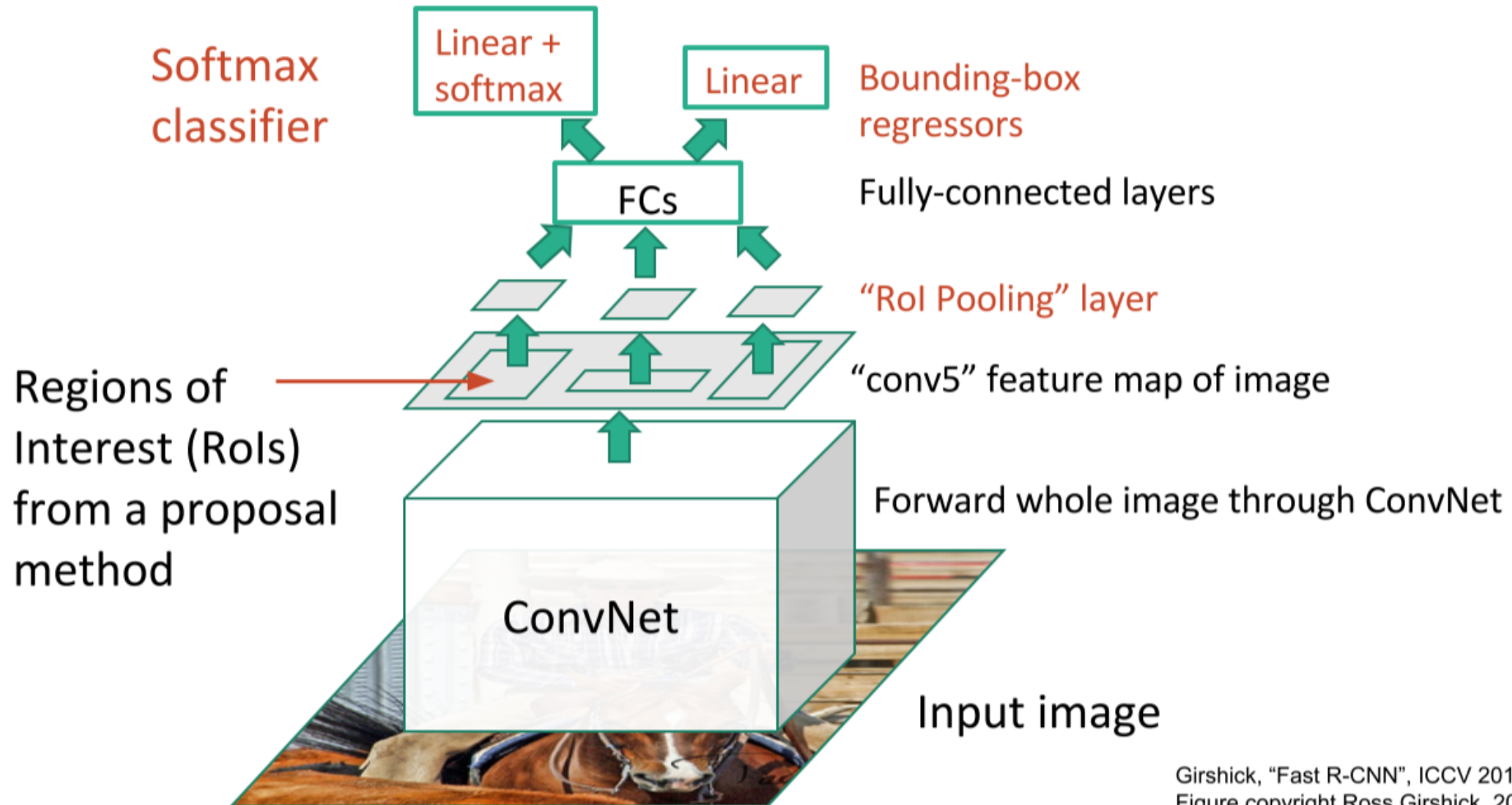


Fast RCNN

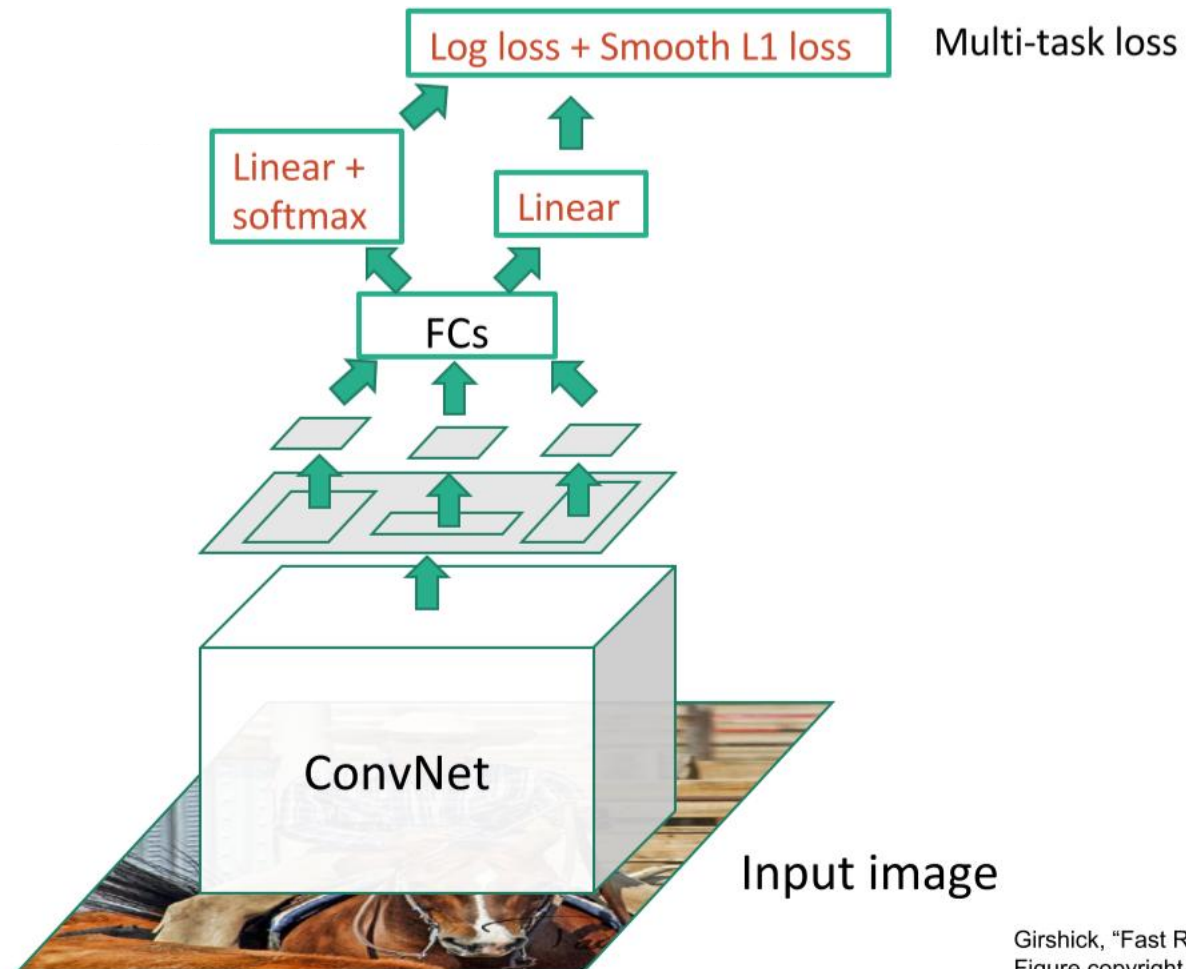


Girshick, "Fast R-CNN", ICCV 2015.
Figure copyright Ross Girshick, 2015; [source](#).

Fast RCNN

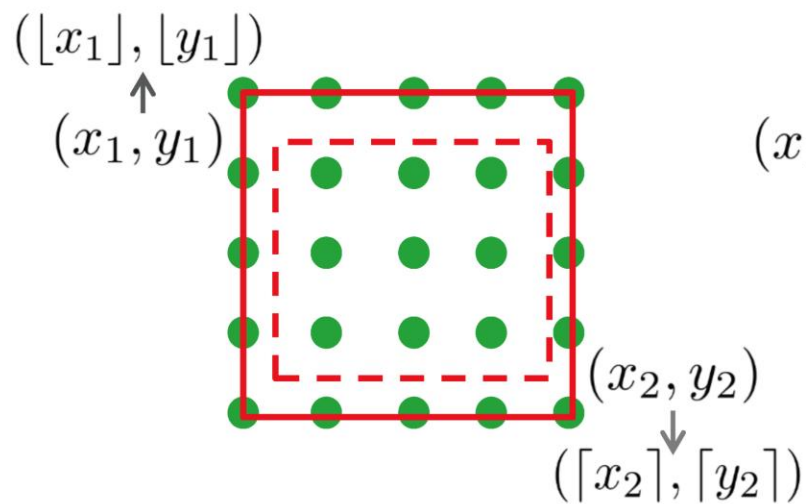


Fast RCNN



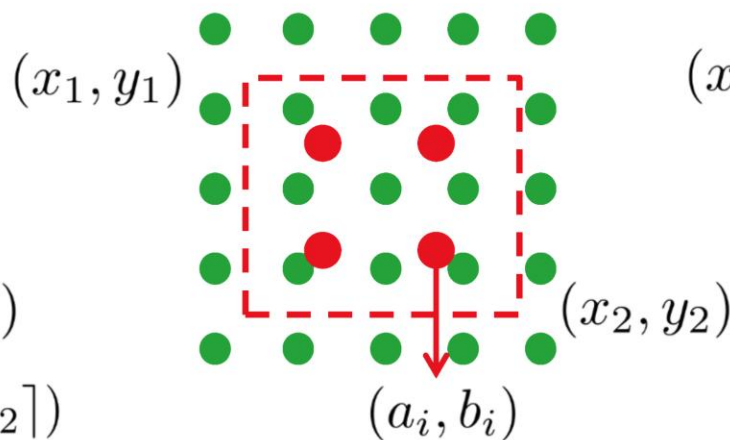
ROI Polling vs Align

1. RoI Pooling



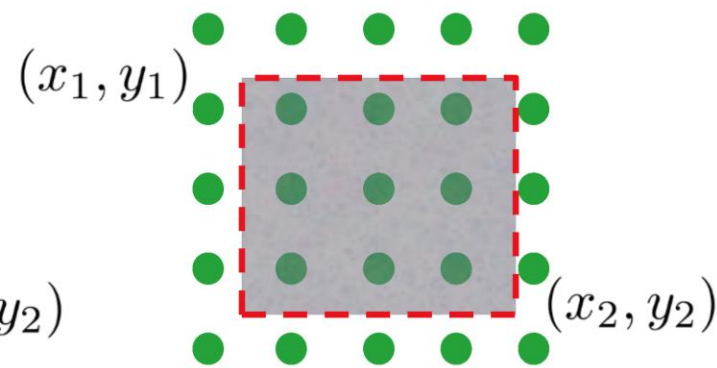
$$\frac{\sum_{i=\lfloor x_1 \rfloor}^{\lceil x_2 \rceil} \sum_{j=\lfloor y_1 \rfloor}^{\lceil y_2 \rceil} w_{i,j}}{(\lceil x_2 \rceil - \lfloor x_1 \rfloor + 1) \times (\lceil y_2 \rceil - \lfloor y_1 \rfloor + 1)}$$

2. RoI Align



$$\sum_{i=1}^N f(a_i, b_i) / N$$

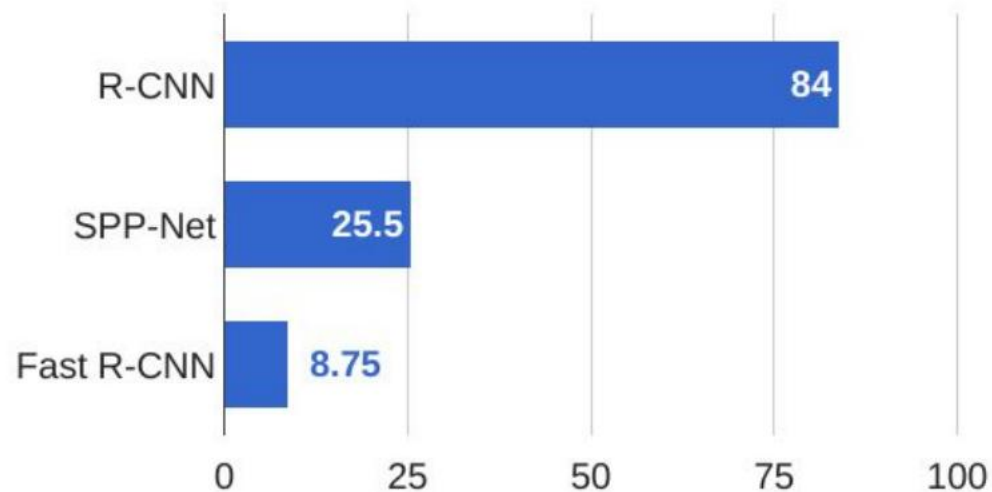
3. PrRoI Pooling



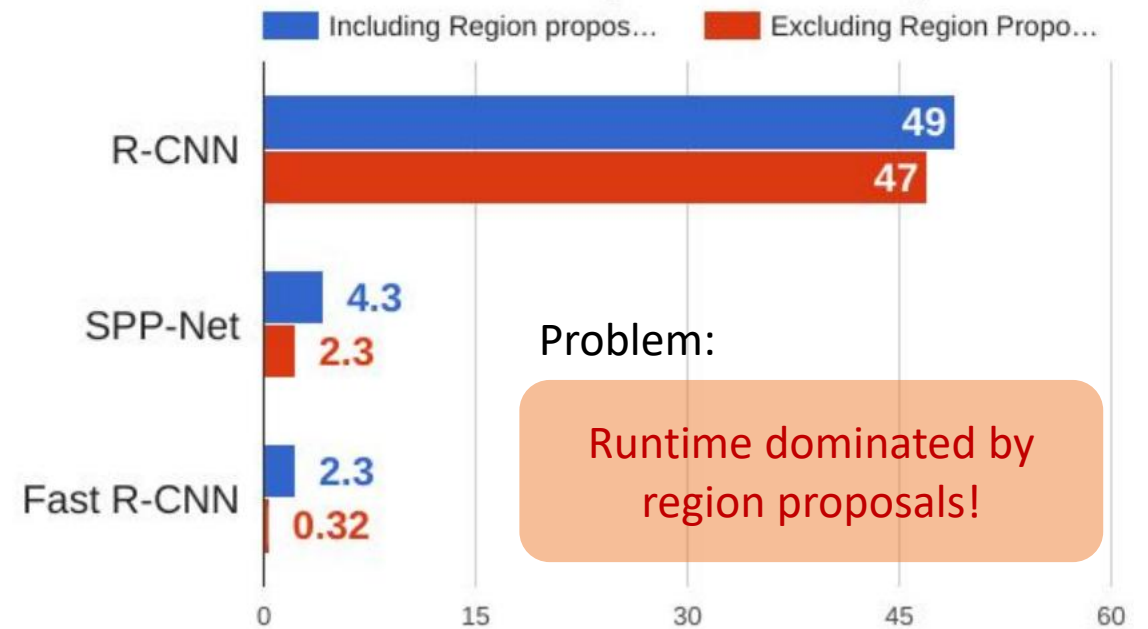
$$\frac{\int_{y_1}^{y_2} \int_{x_1}^{x_2} f(x, y) dx dy}{(x_2 - x_1) \times (y_2 - y_1)}$$

RCNN vs Fast RCNN

Training time (Hours)



Test time (seconds)

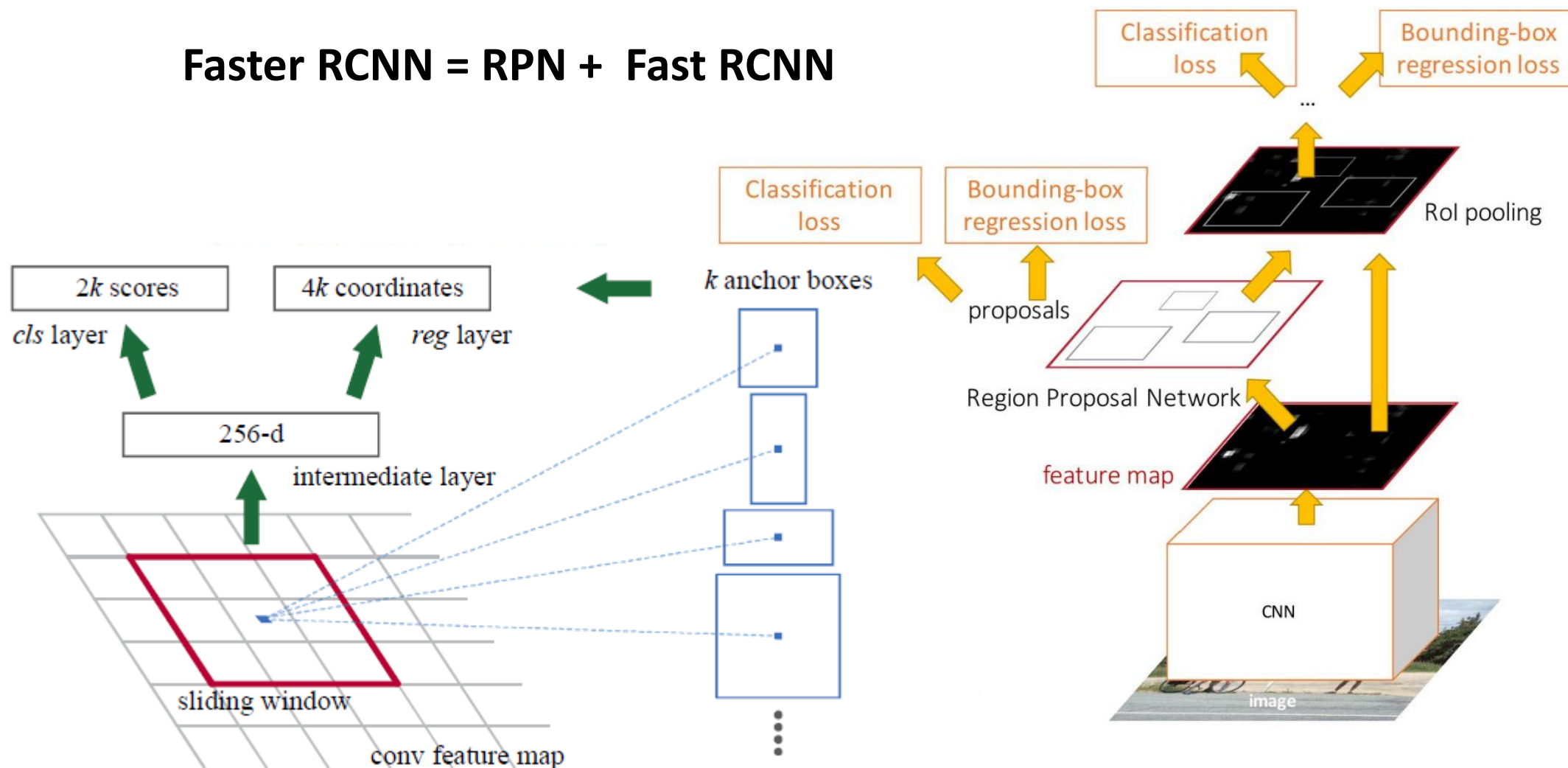


Problem:

Runtime dominated by
region proposals!

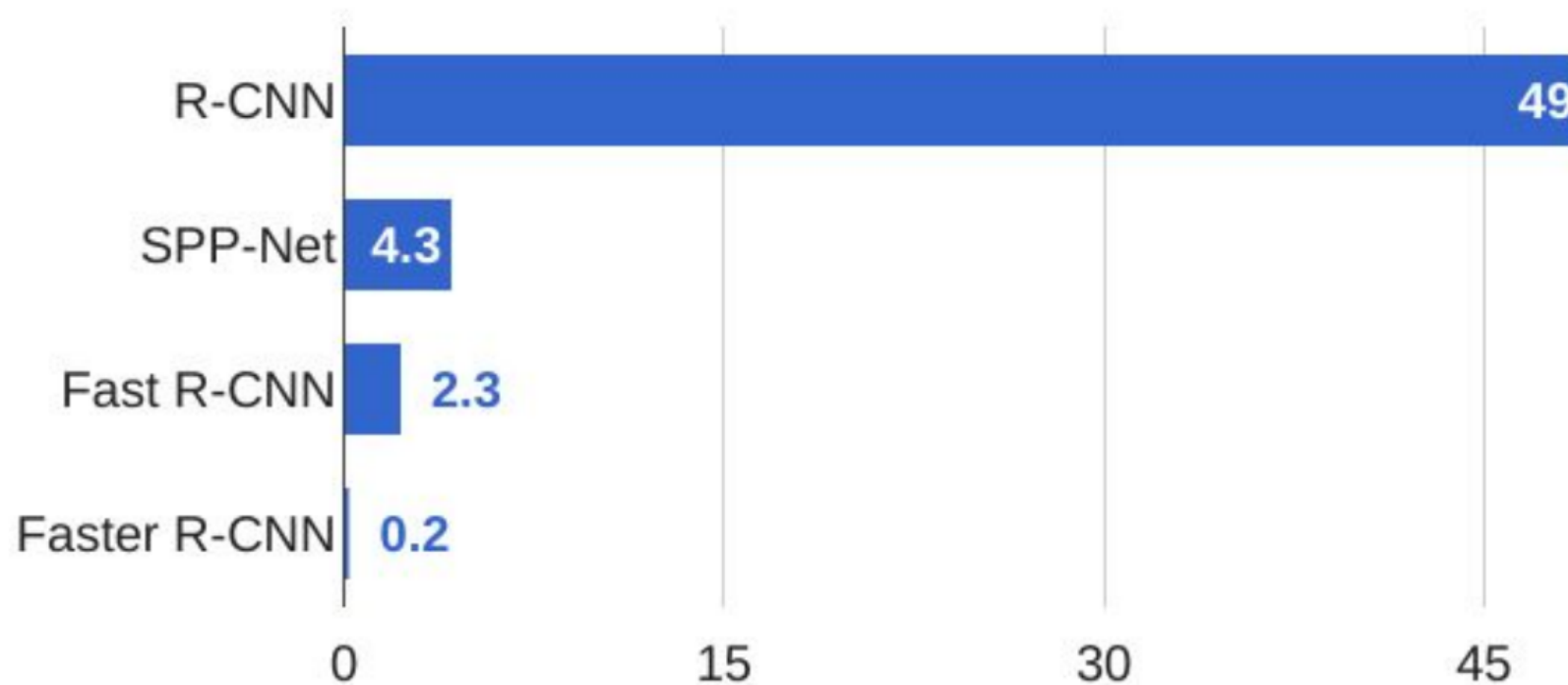
Fast^{er} RCNN

Faster RCNN = RPN + Fast RCNN

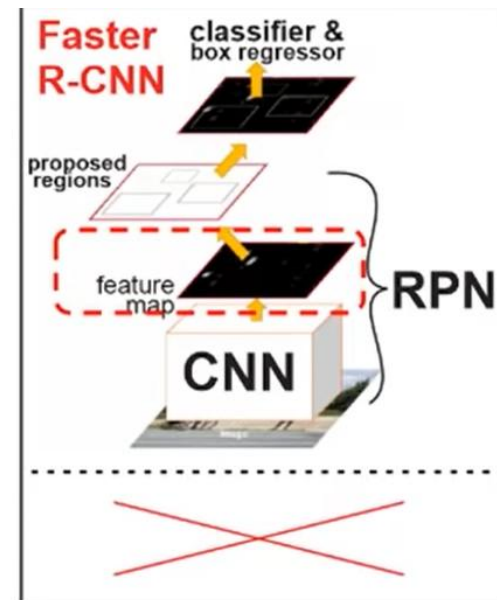
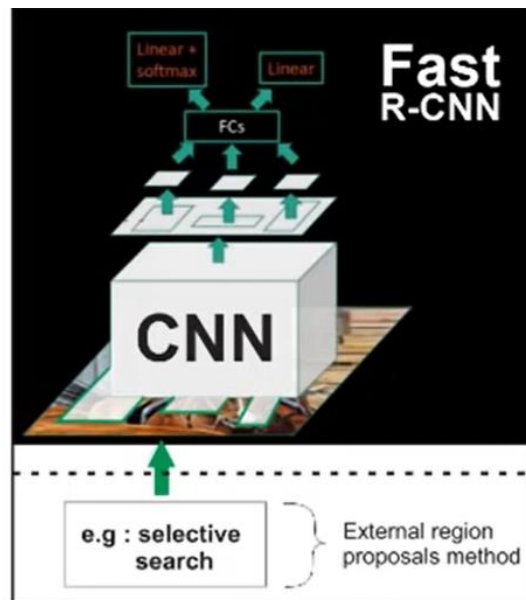
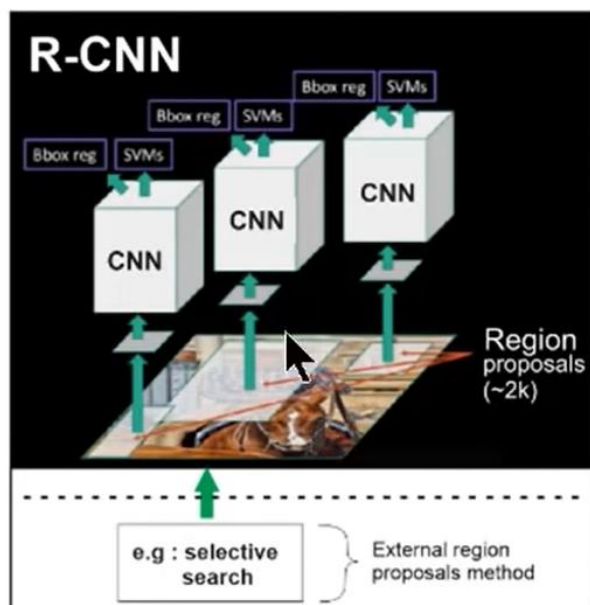


Fast^{er} RCNN

R-CNN Test-Time Speed



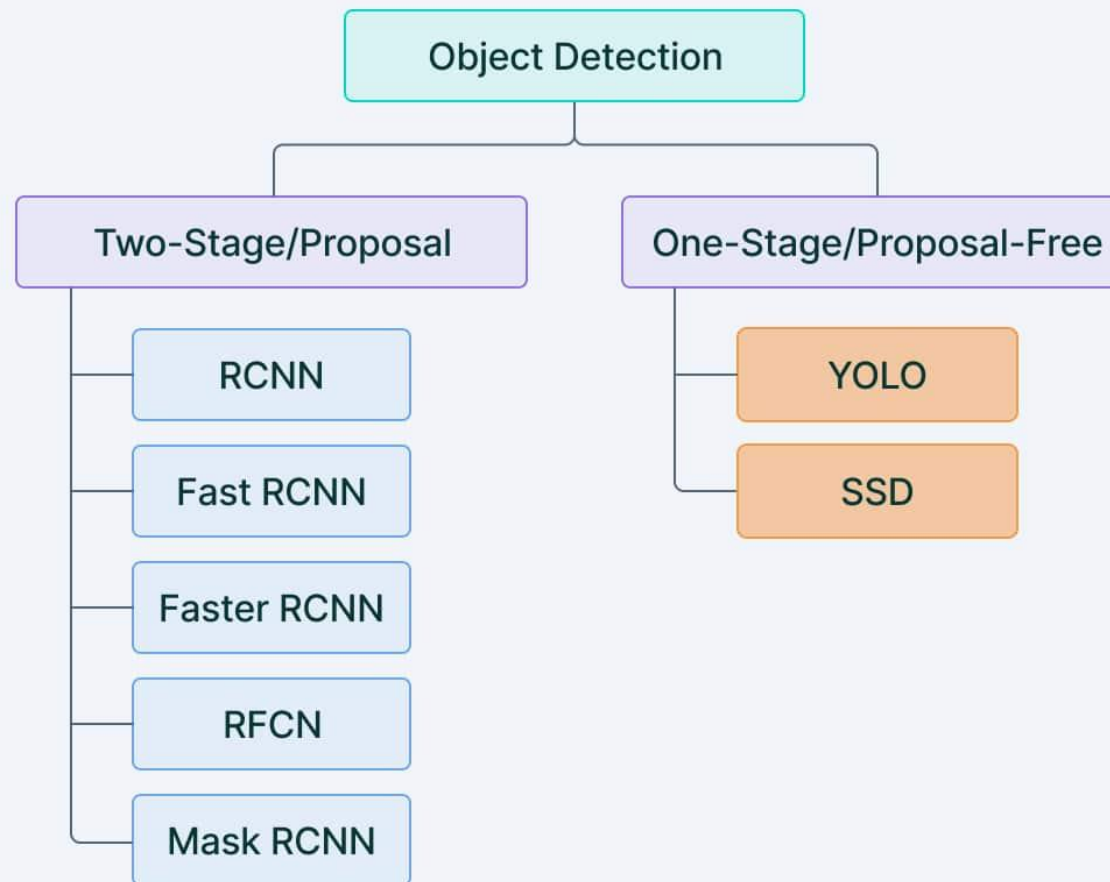
Summary



	R-CNN	Fast R-CNN	Faster R-CNN
Test time per image	50 seconds	2 seconds	0.2 seconds
Speed-up	1x	25x	250x
mAP (VOC 2007)	66.0%	66.9%	66.9%

Overview

One and two stage detectors



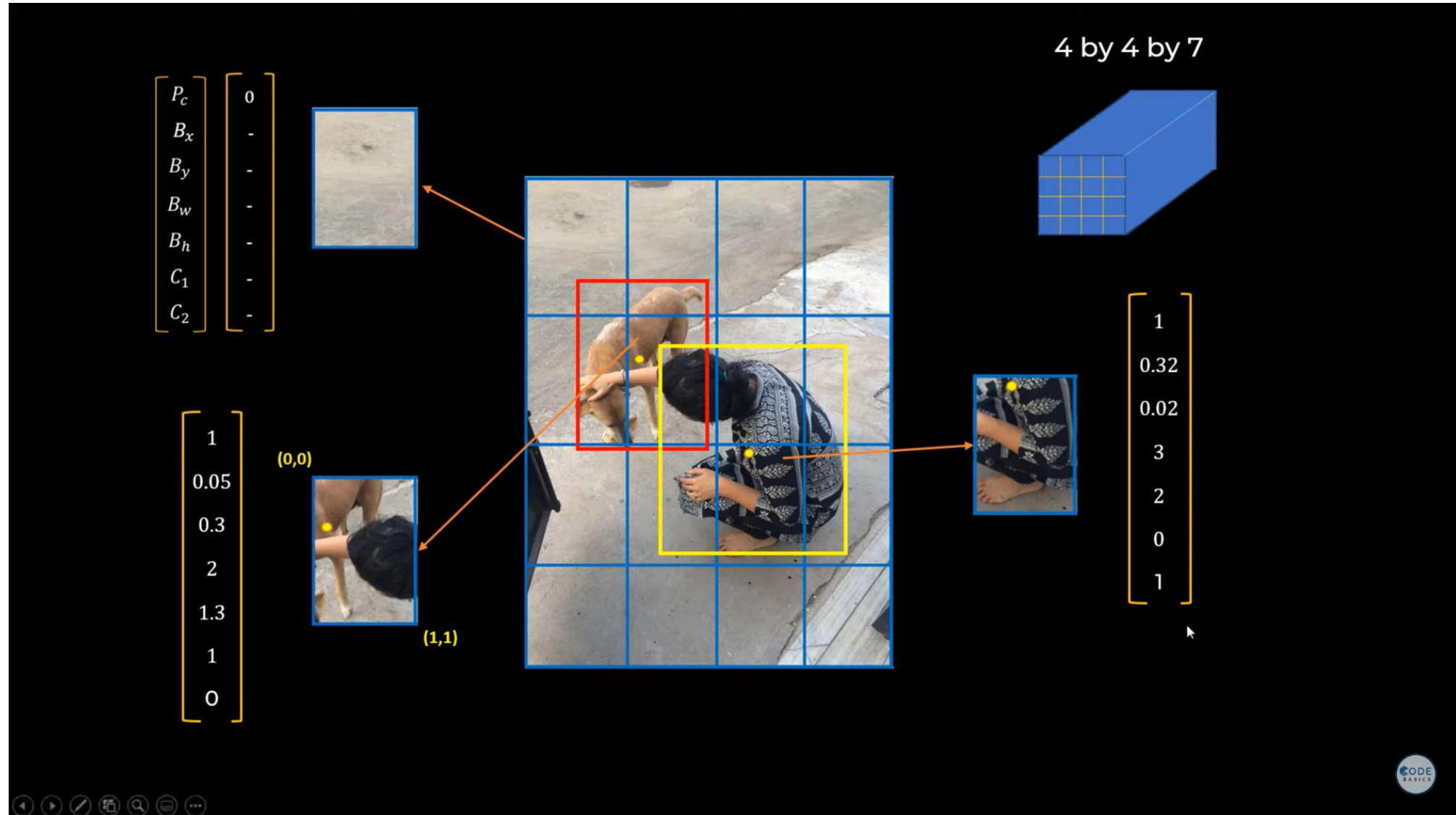
Introduction to YOLO



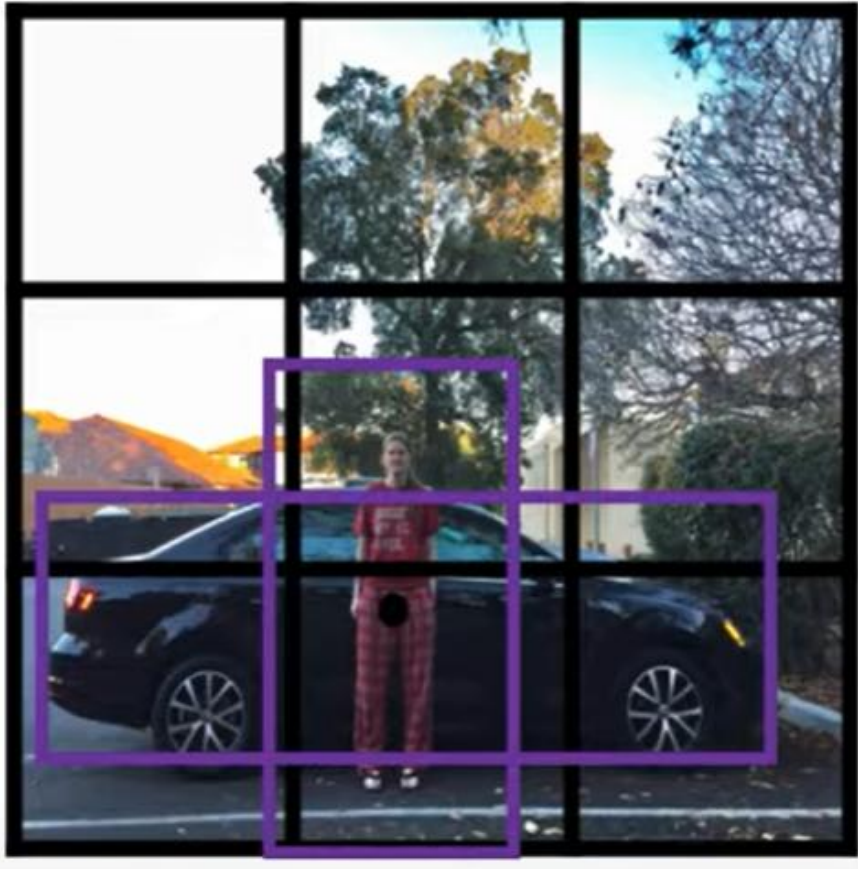
P_c	1
B_x	50
B_y	70
B_w	60
B_h	70
C_1	1
C_2	0

$C_1 = \text{Dog class}$
 $C_2 = \text{Person Class}$

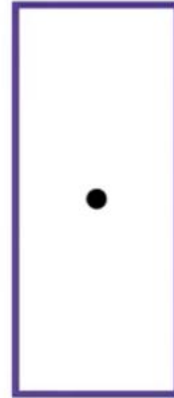
Introduction to YOLO



Overlapping Objects



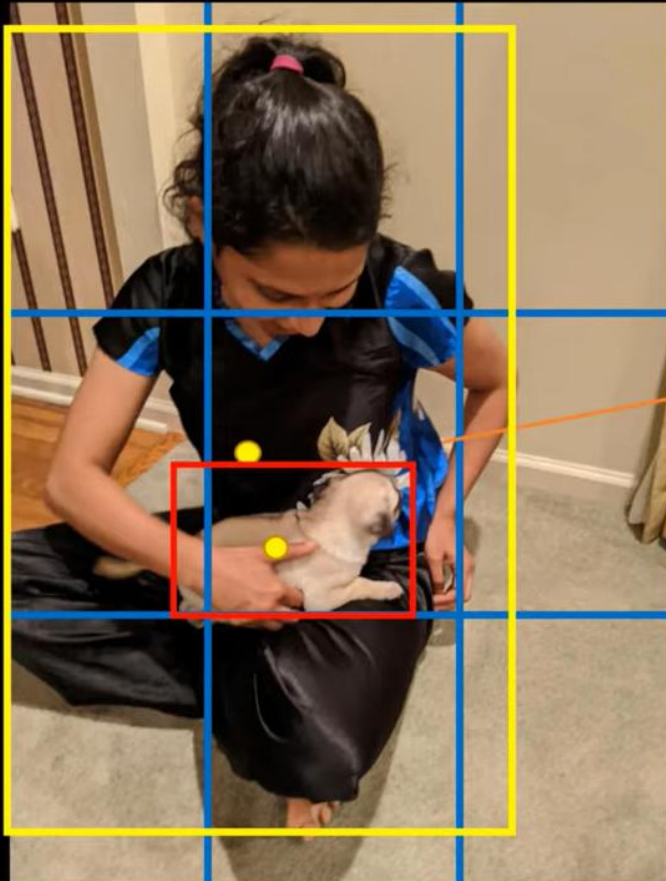
Anchor box 1:



Anchor box 2:



Overlapping Objects



P_c
 B_x
 B_y
 B_w
 B_h
 C_1
 C_2

1
0.22
0.45
1
0.7
1
0

Dog

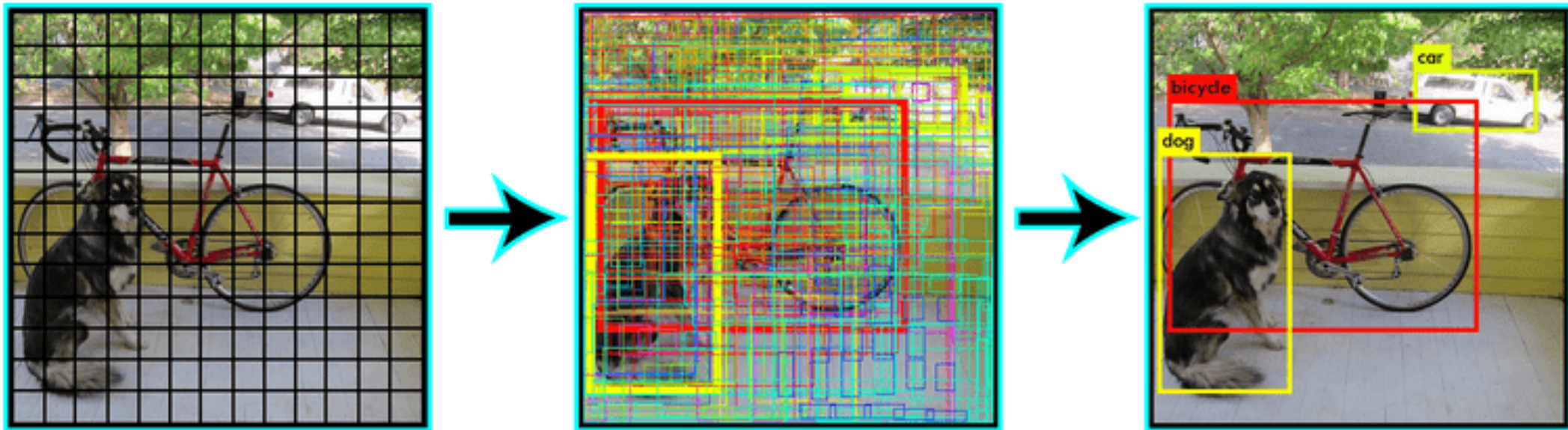
1
0.13
0.38
2.1
3
0
1

Person

1
0.22
0.45
1
0.7
1
0
1
0.13
0.38
2.1
3
0
1

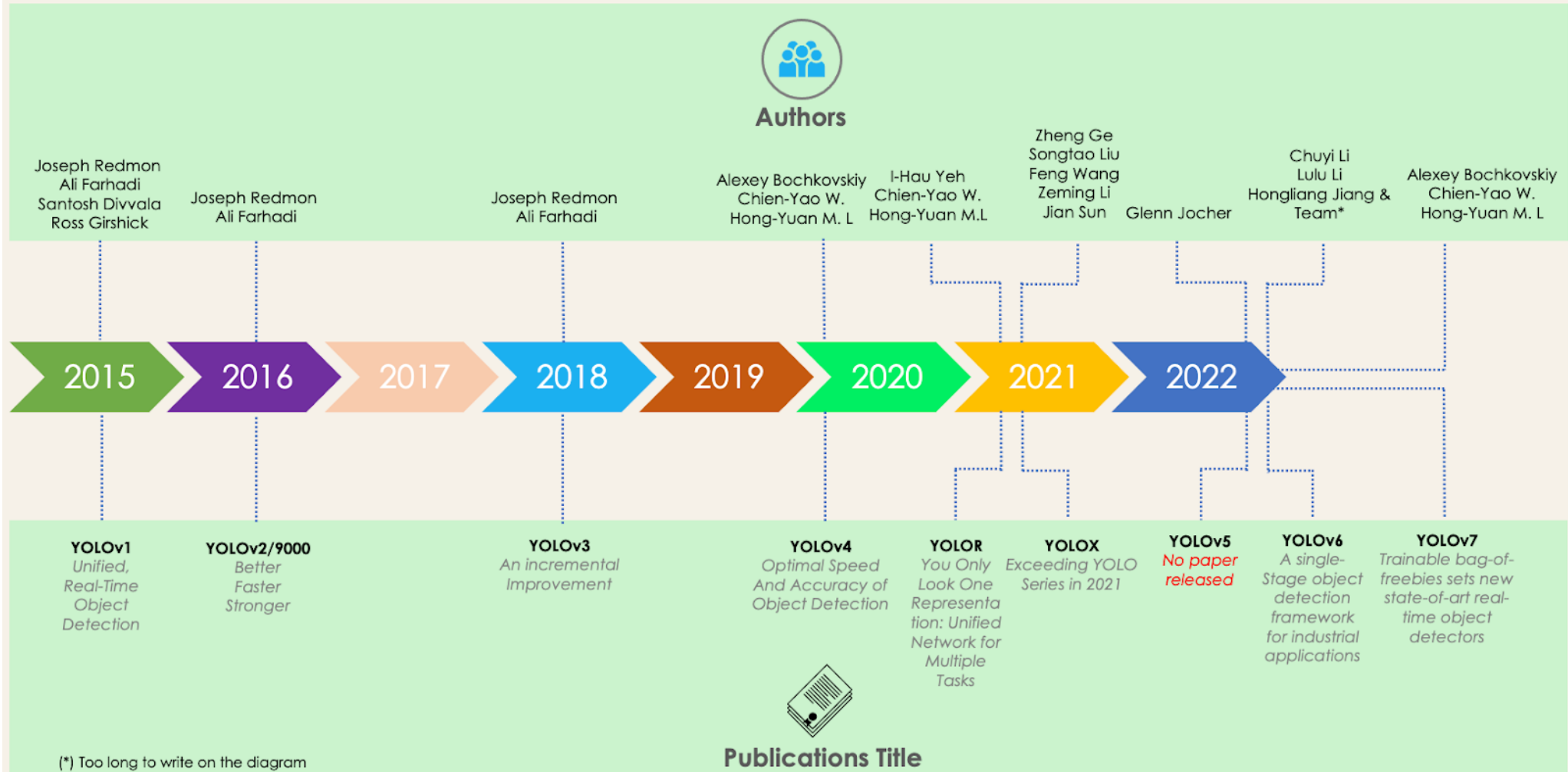
YOLO: You Look Only Once

- Single Stage
- Super Fast



YOLO Timeline

YOLO Timeline From 2015 to 2022



Morality!



Joseph Redmon
@pjreddie



I stopped doing CV research because I saw the impact my work was having. I loved the work but the military applications and privacy concerns eventually became impossible to ignore.



Roger Grosse @RogerGrosse · Feb 20, 2020

Replying to @skoularidou

What's an example of a situation where you think someone should decide not to submit their paper due to Broader Impacts reasons?

7:39 PM · Feb 20, 2020

YOLO : intuition



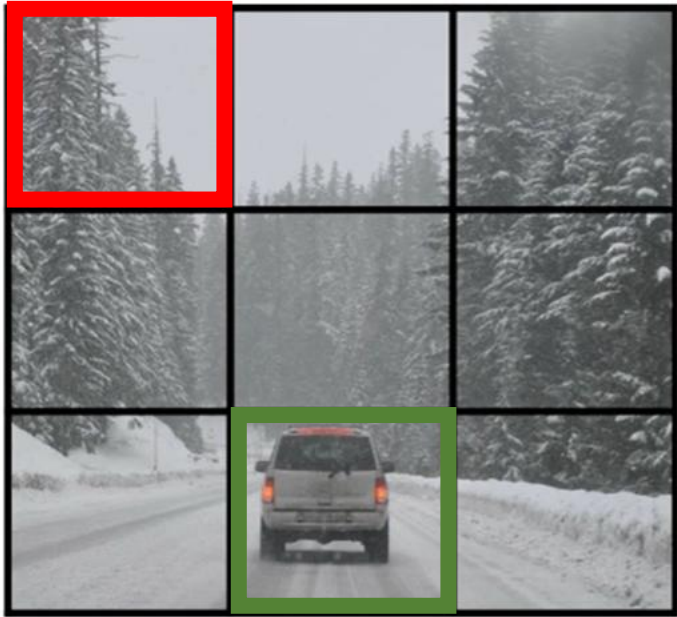
- 1 - pedestrian
- 2 - car
- 3 - motorcycle

y is

?

$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

YOLO : intuition



...

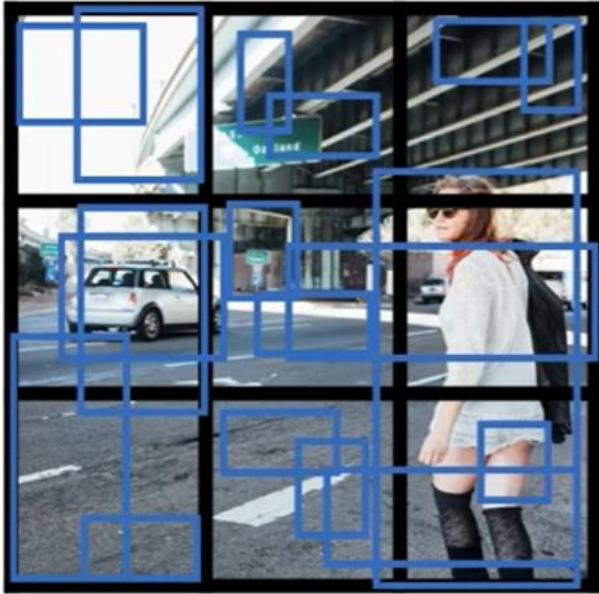


$3 \times 3 \times 2 \times 8$

$y =$

$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

NMS



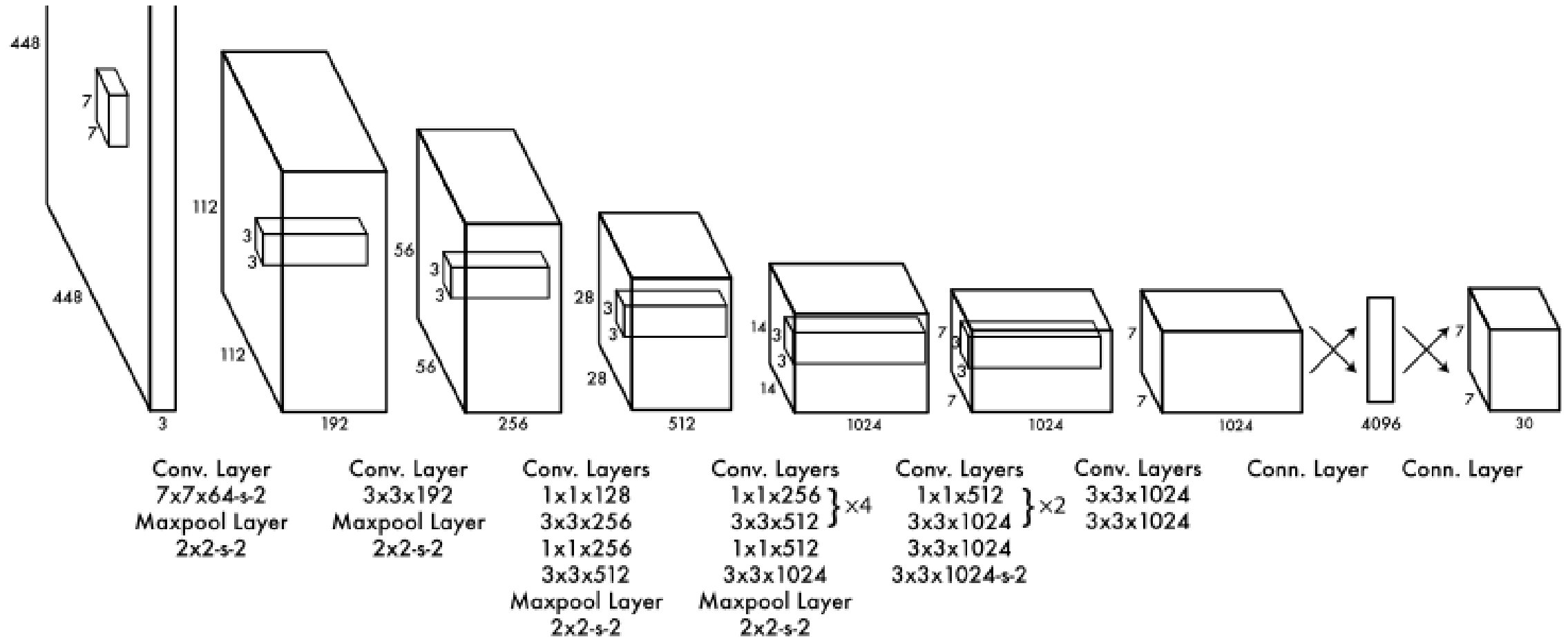
- For each grid call, get 2 predicted bounding boxes.

NMS

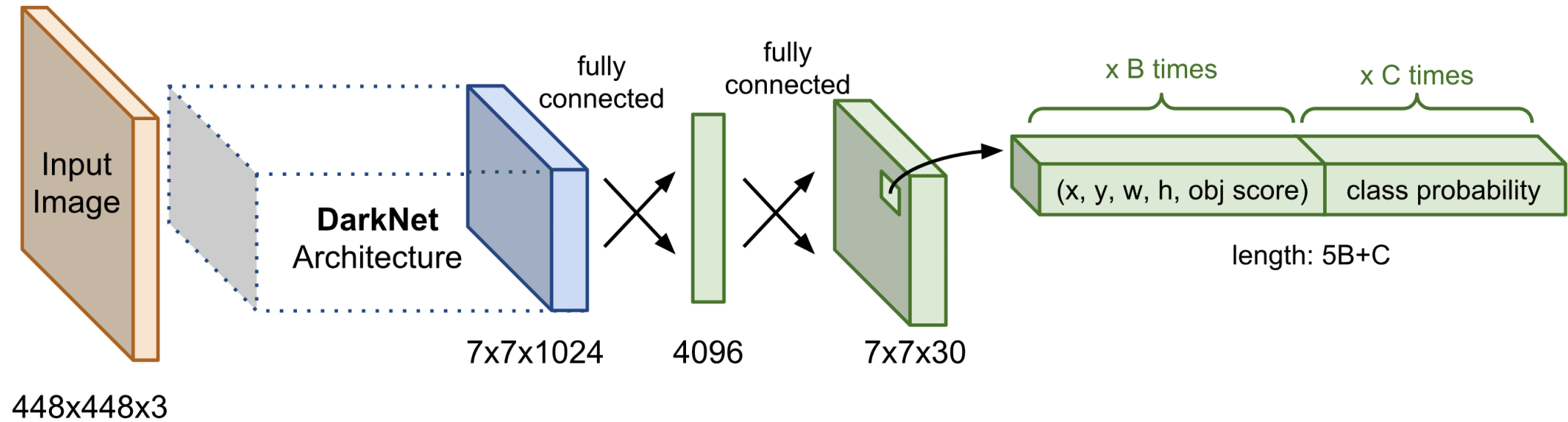


- For each grid cell, get 2 predicted bounding boxes.
- Get rid of low probability predictions.
- For each class (pedestrian, car, motorcycle) use non-max suppression to generate final predictions.

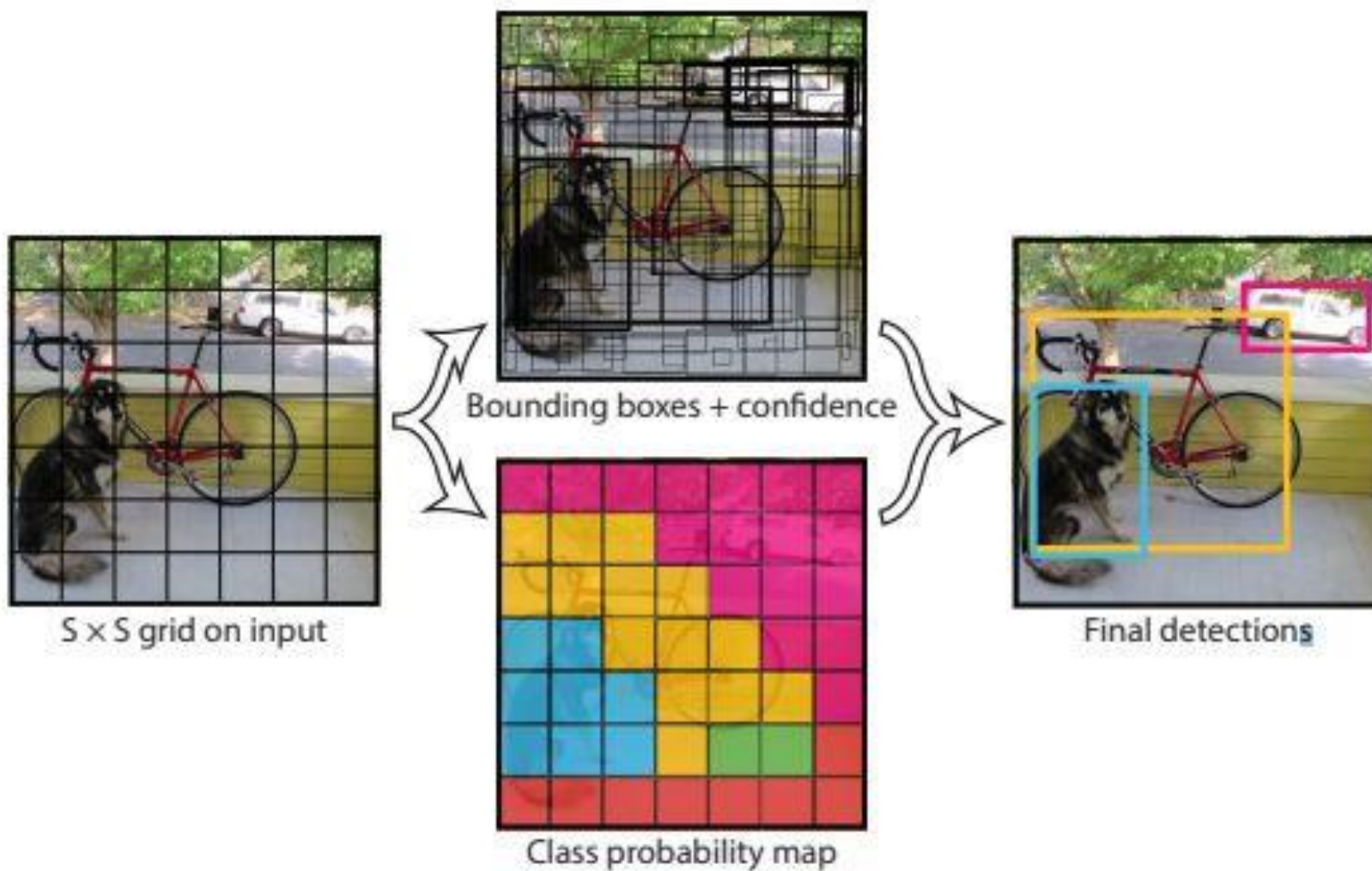
YOLO Architecture



YOLO Architecture



YOLO Algorithm



YOLO (Test time)



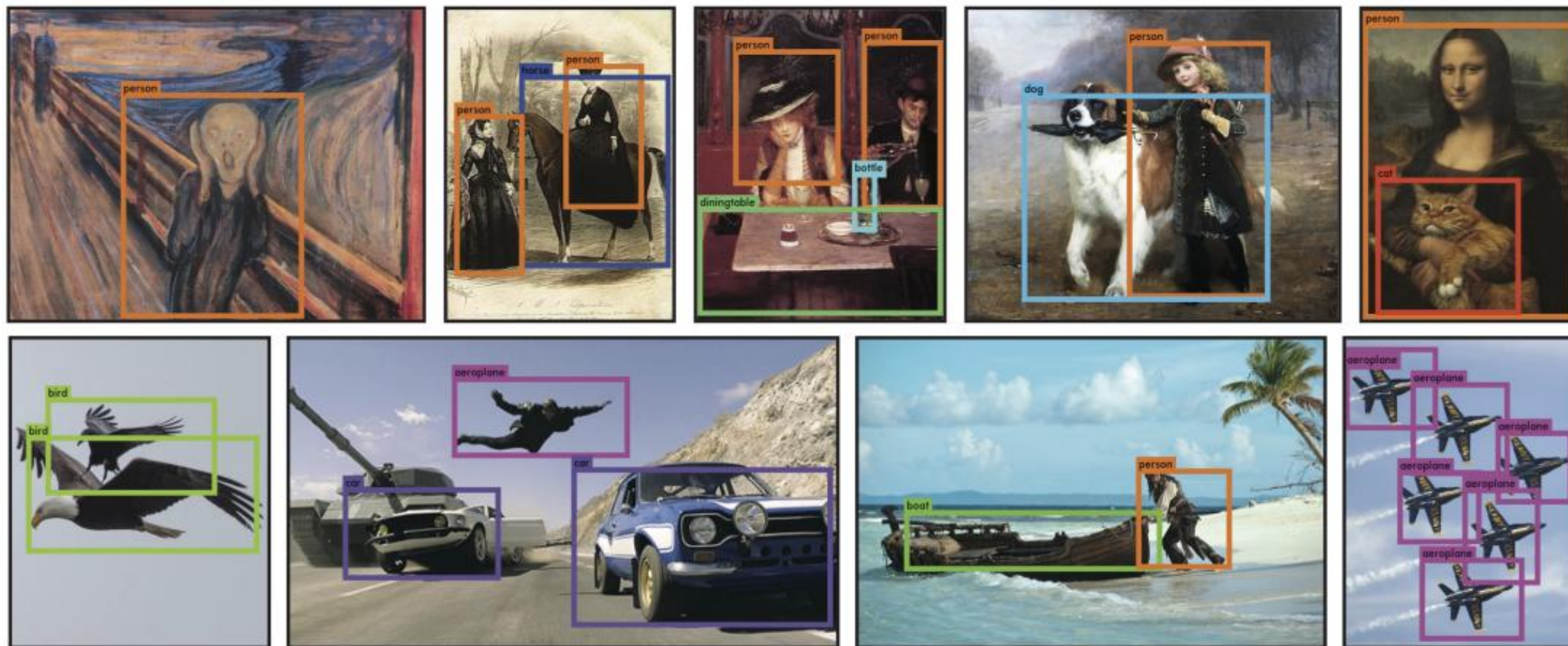
YOLO (Test time)



YOLO loss function

$$\begin{aligned} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2 \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2 \\ & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \end{aligned}$$

Generalization



Limitations of YOLO

- Small Objects
- Localization of bounding box
- Etc.



YOLO v2/9000

YOLO9000: Better, Faster, Stronger

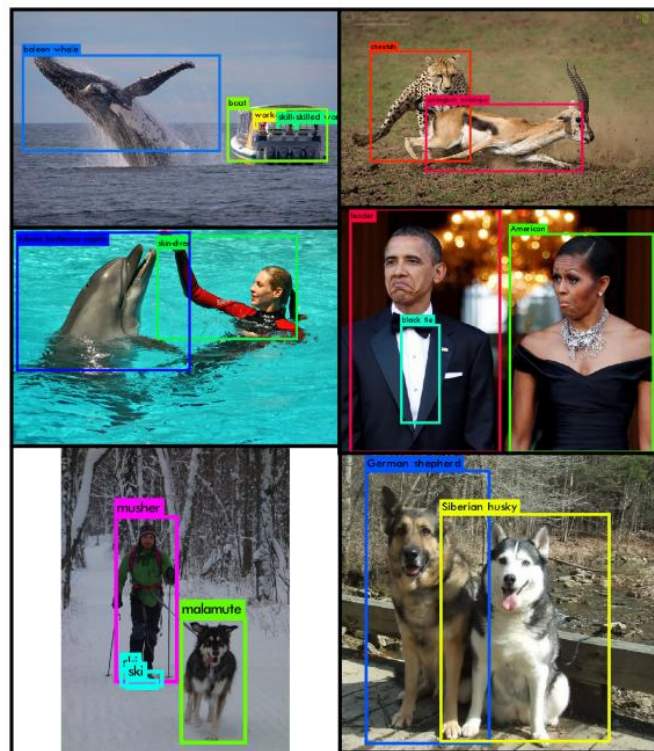
Joseph Redmon^{*†}, Ali Farhadi^{*†}

University of Washington^{*}, Allen Institute for AI[†]

<http://pjreddie.com/yolo9000/>

Abstract

We introduce YOLO9000, a state-of-the-art, real-time object detection system that can detect over 9000 object categories. First we propose various improvements to the YOLO detection method, both novel and drawn from prior work. The improved model, YOLOv2, is state-of-the-art on standard detection tasks like PASCAL VOC and COCO. Using a novel, multi-scale training method the same YOLOv2 model can run at varying sizes, offering an easy tradeoff between speed and accuracy. At 67 FPS, YOLOv2 gets 76.8 mAP on VOC 2007. At 40 FPS, YOLOv2 gets 78.6 mAP, outperforming state-of-the-art methods like Faster R-CNN with ResNet and SSD while still running significantly faster. Finally we propose a method to jointly train on object detection and classification. Using this method we train YOLO9000 simultaneously on the COCO detection dataset and the ImageNet classification dataset. Our joint training allows YOLO9000 to predict detections for object classes that don't have labelled detection data. We validate our approach on the ImageNet detection task. YOLO9000 gets 19.7 mAP on the ImageNet detection validation set despite only having detection data for 44 of the 200 classes. On the 156 classes not in COCO, YOLO9000 gets 16.0 mAP. But

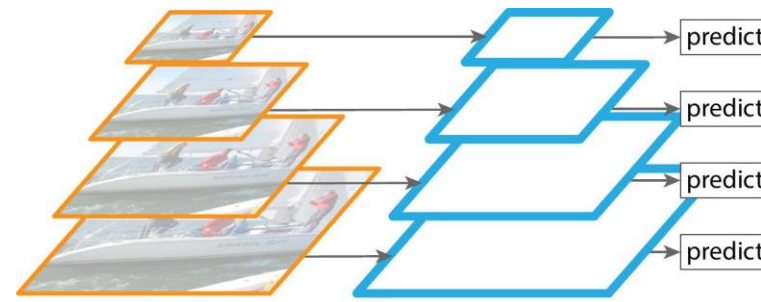
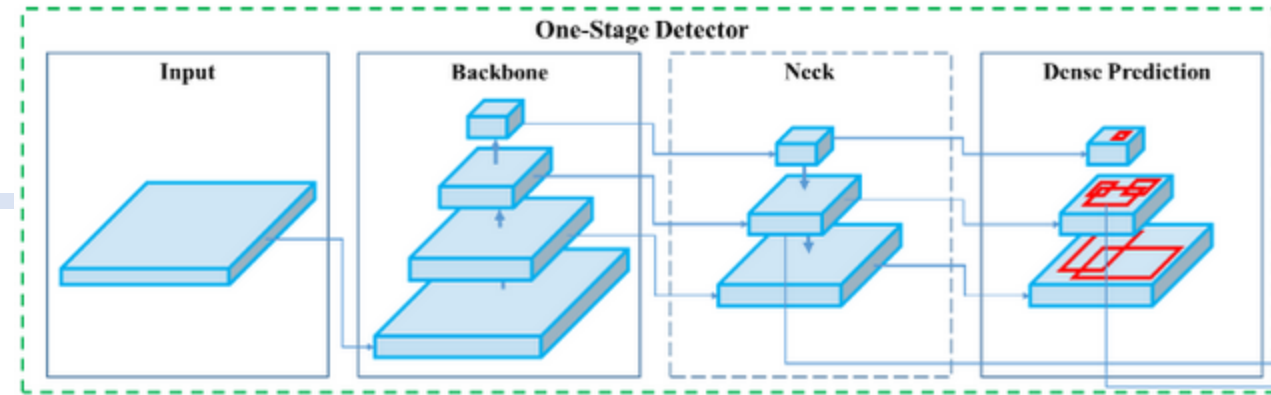


YOLO v3

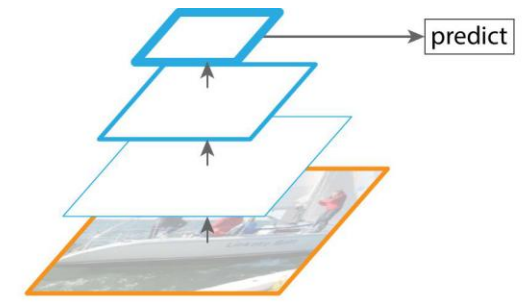
- Stronger, Faster but not Better!
- Darknet-53
- Anchor boxes with different scales and aspect ratios

- introduced the concept of "feature pyramid networks" (FPN)

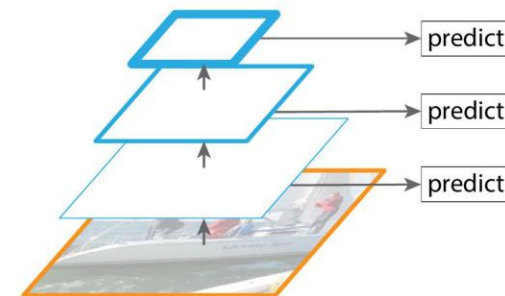
- FPNs are a CNN architecture used to detect objects at multiple scales



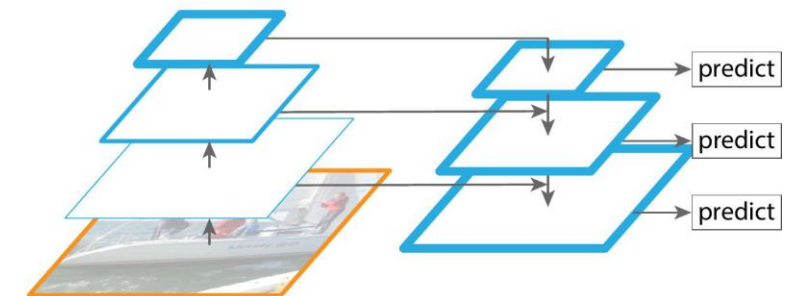
(a) Featurized image pyramid



(b) Single feature map

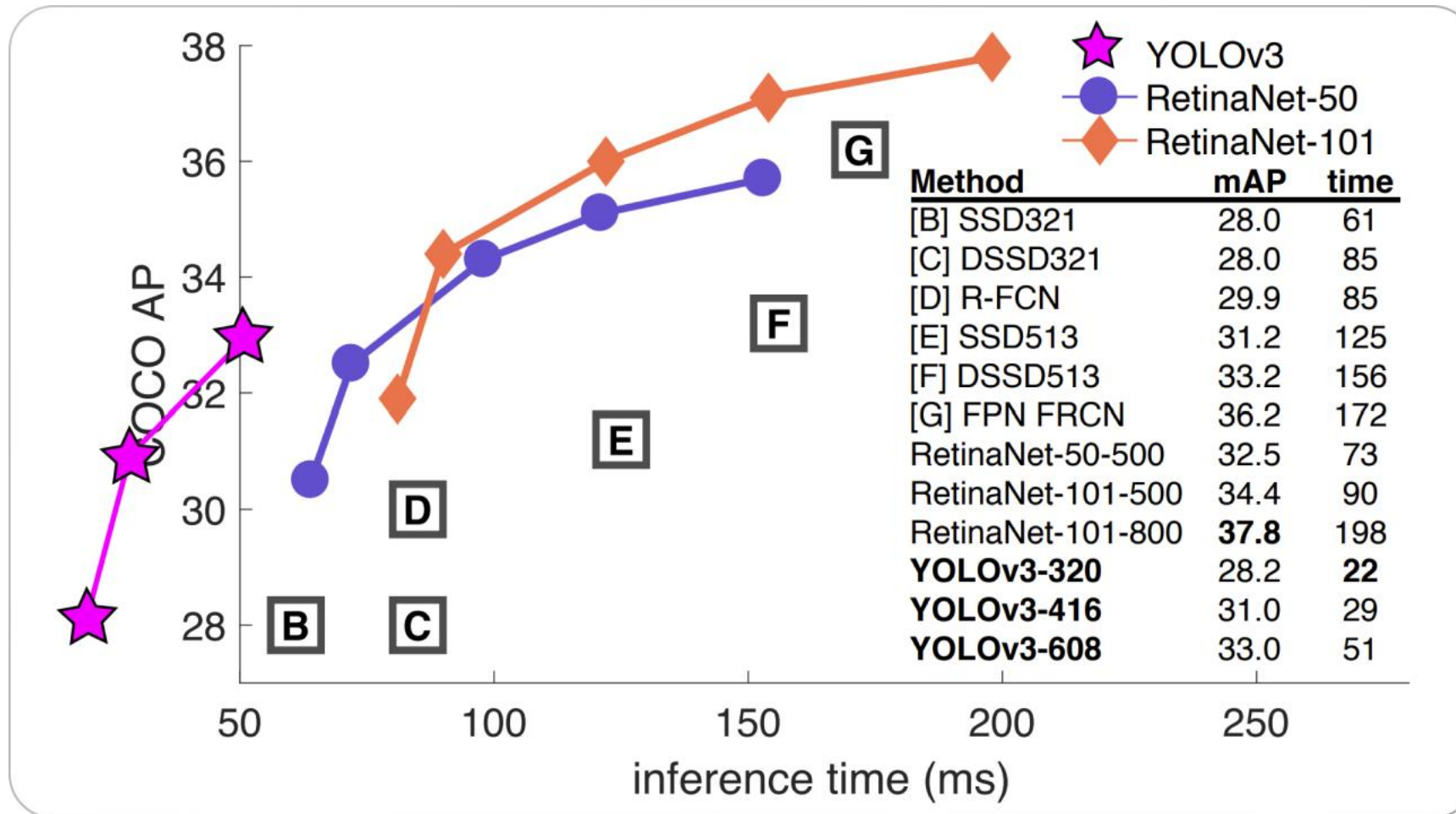


(c) Pyramidal feature hierarchy



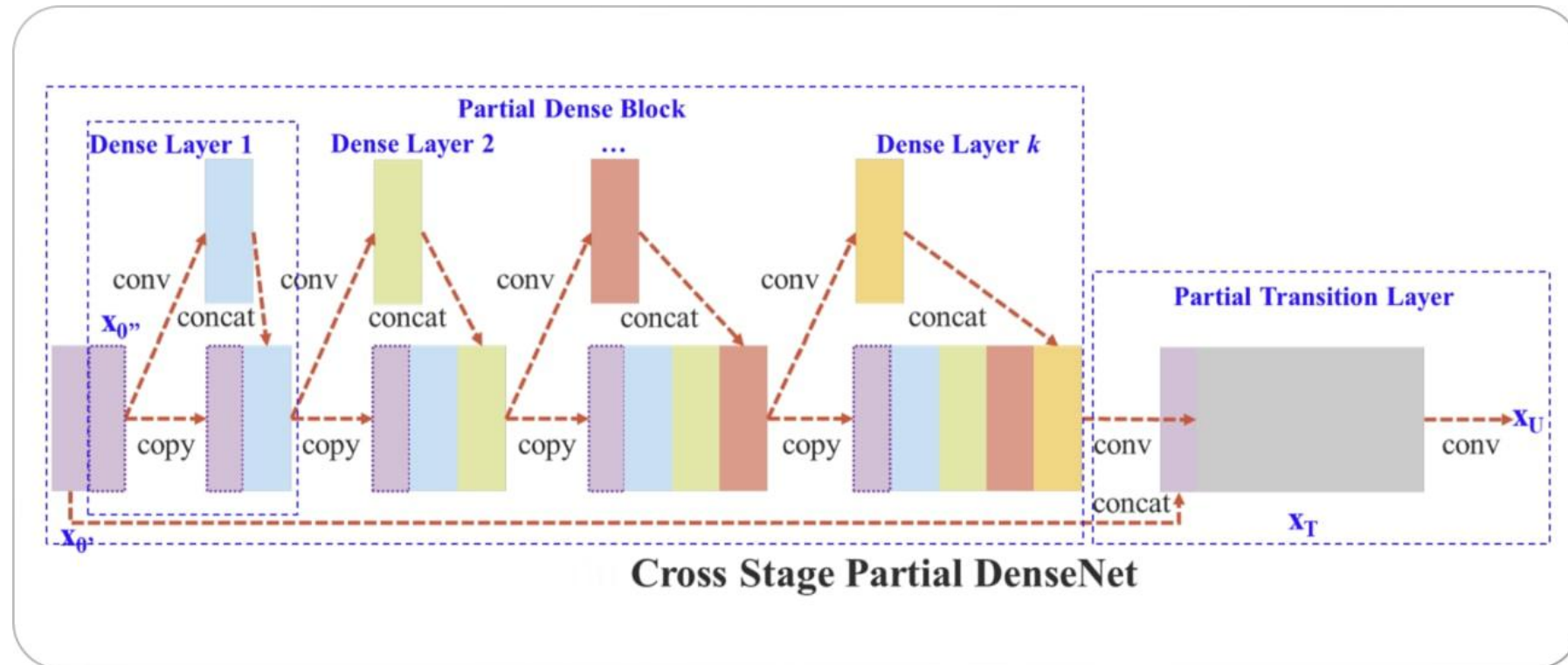
(d) Feature Pyramid Network

YOLO v3 Result



YOLO v4

- CSPDarknet53 (Cross Stage Partial Network)
- A new method for generating the anchor boxes, called "k-means clustering
- New term in loss function (GHM loss)
- Bag of Freebies
- Mish activation function
- DIoU NMS



YOLO v4 Result

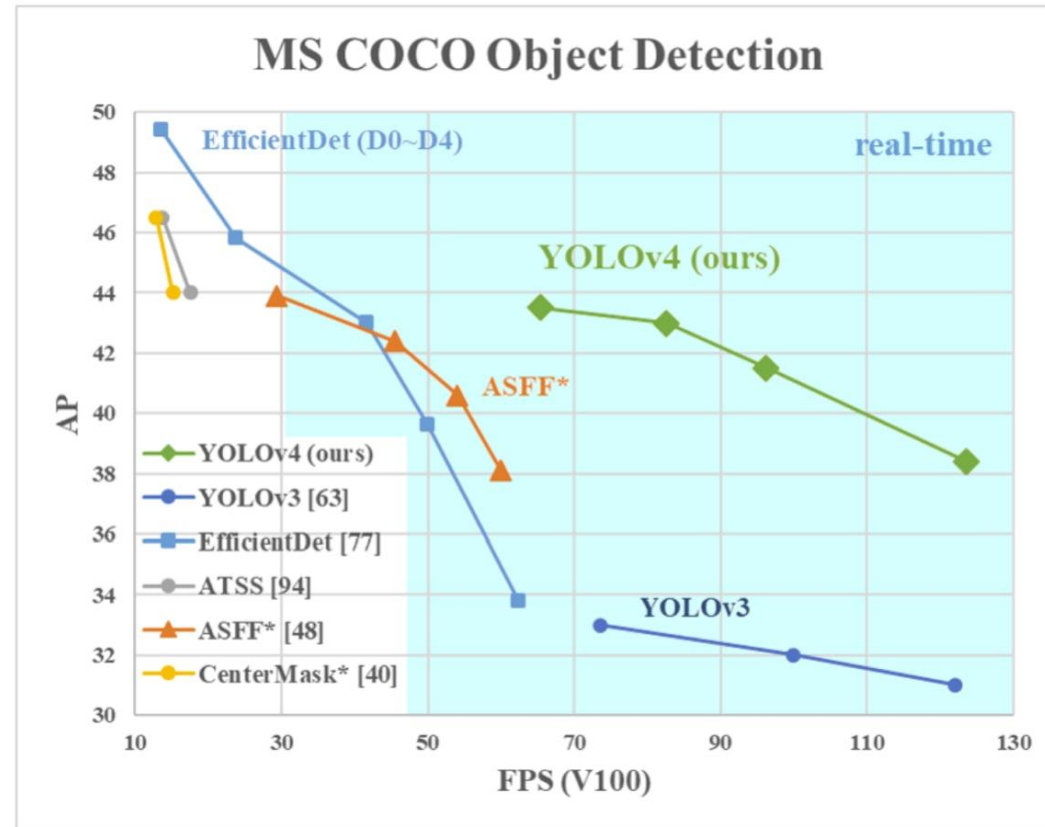
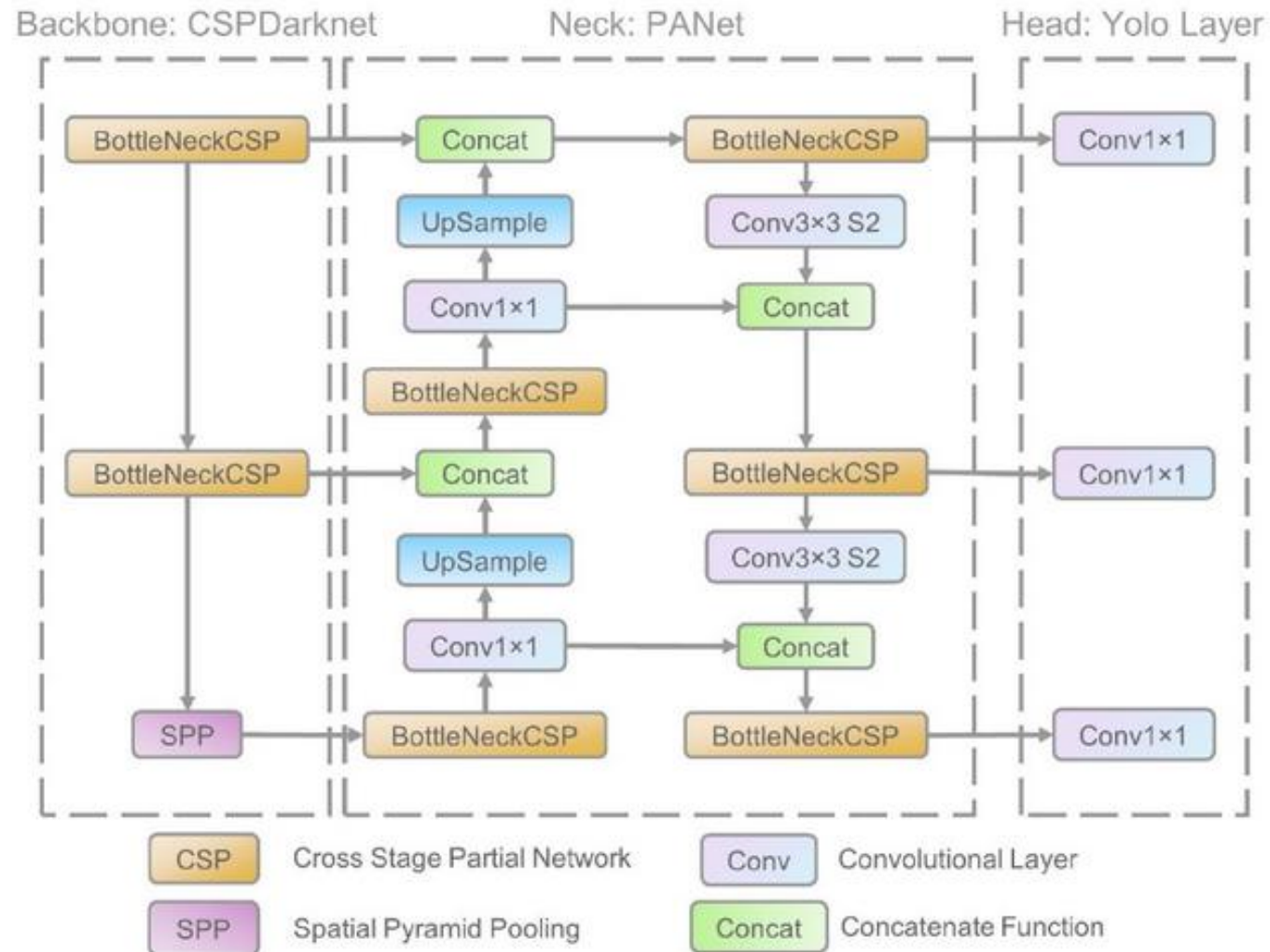
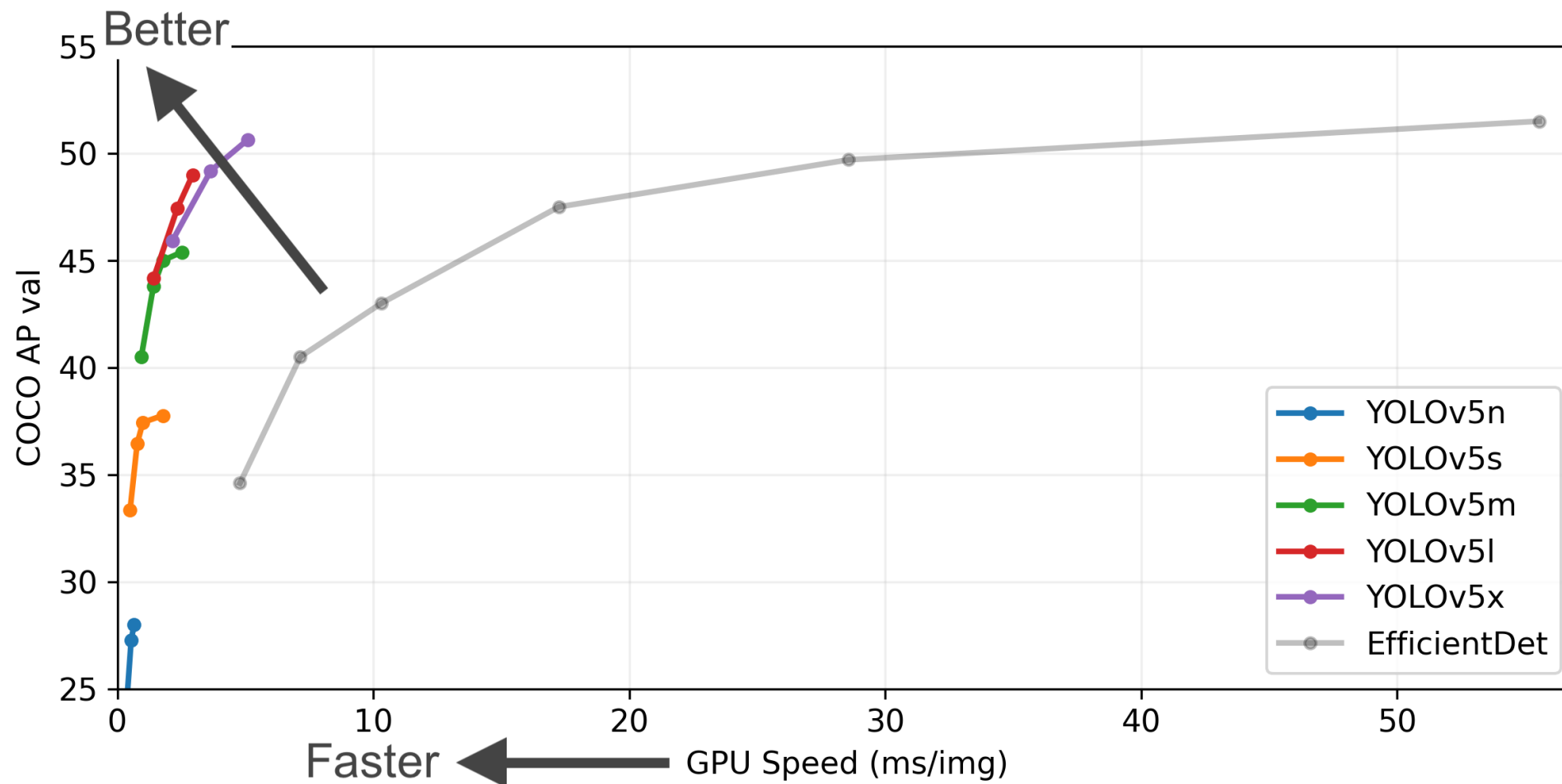


Figure 1: Comparison of the proposed YOLOv4 and other state-of-the-art object detectors. YOLOv4 runs twice faster than EfficientDet with comparable performance. Improves YOLOv3's AP and FPS by 10% and 12%, respectively.

YOLO v5



YOLO v5



YOLO v5

Model	size (pixels)	mAP ^{val} 50-95	mAP ^{val} 50	Speed CPU b1 (ms)	Speed V100 b1 (ms)	Speed V100 b32 (ms)	params (M)	FLOPs @640 (B)
YOLOv5n	640	28.0	45.7	45	6.3	0.6	1.9	4.5
YOLOv5s	640	37.4	56.8	98	6.4	0.9	7.2	16.5
YOLOv5m	640	45.4	64.1	224	8.2	1.7	21.2	49.0
YOLOv5l	640	49.0	67.3	430	10.1	2.7	46.5	109.1
YOLOv5x	640	50.7	68.9	766	12.1	4.8	86.7	205.7
YOLOv5n6	1280	36.0	54.4	153	8.1	2.1	3.2	4.6
YOLOv5s6	1280	44.8	63.7	385	8.2	3.6	12.6	16.8
YOLOv5m6	1280	51.3	69.3	887	11.1	6.8	35.7	50.0
YOLOv5l6	1280	53.7	71.3	1784	15.8	10.5	76.8	111.4
YOLOv5x6	1280	55.0	72.7	3136	26.2	19.4	140.7	209.8
+ TTA	1536	55.8	72.7	-	-	-	-	-

Keywords

- SSD
- Focal loss
- Retina Net
- Efficient Det
- YOLO v4
- YOLO v5
- YOLO V6
- YOLO V7
- YOLO V8
- Scaled-YOLOv4
- PP-YOLO
- YOLOX
- YOLOR
- Robo flow